TOWARD RELIABLE MACHINE LEARNING: UNDERSTANDING, EXPLAINING, AND REPAIRING MODEL PREDICTIONS

by

KRISHNA KHADKA

Dissertation Proposal

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THE UNIVERSITY OF TEXAS AT ARLINGTON

April 30, 2026

Supervising Professor: Dr. Yu Lei

Committee Members:

Dr. Allison Sullivan
Dr. Christoph Csallner
Dr. Junzhou Huang

Abstract

TOWARD RELIABLE MACHINE LEARNING:
UNDERSTANDING, EXPLAINING, AND REPAIRING MODEL PREDICTIONS

Krishna Khadka, Ph.D. (Proposed)
Supervising Professor: Yu Lei

Machine learning models have achieved remarkable predictive performance but remain largely opaque, raising critical concerns about reliability in high-stakes applications. This dissertation addresses this challenge through a progression of four contributions that move from understanding model predictions to explaining them faithfully to repairing systematic failures.

This dissertation is presented in an article-based format and comprises four research papers. The first paper empirically validates that most model predictions are determined by interactions among only a few features. The second paper introduces ABLE, a novel approach that generates adversarial pairs bracketing the decision boundary to construct local explanations with significantly higher fidelity and stability than existing methods. The third paper presents DD-CAM, which adapts delta debugging from software engineering to identify the minimal sufficient set of representational units that preserves a vision model's prediction, producing focused saliency maps. The fourth paper proposes EGMR, an explanation-guided framework that identifies recurring failure-driving concepts and repairs shortcut correlations through targeted synthetic augmentation that fills under-represented cells of the joint concept-class distribution, without requiring group labels.

Contents

Abstract **1**

1 Introduction **5**
1.1 Research Overview 5
1.2 Summary of Publications 6

2 Assessing the Degree of Feature Interactions that Determine a Model Prediction **7**
2.1 Introduction 7
2.2 Background 9
2.2.1 Machine Learning Tasks 9
2.2.2 Decision Making in Classification Task 9
2.2.3 SHAP Values 10
2.3 Approach 11
2.3.1 Step 1: Compute and Sort SHAP Values 13
2.3.2 Step 2: Margin-based Iterative Top t Feature Identification (Lines 18-28 of Algorithm 1) 14
2.3.3 Example 14
2.4 Experiment and Results 15
2.4.1 Datasets: 15
2.4.2 Classification Models: 16
2.4.3 SHAP Value Computation 16
2.4.4 Results and Discussion 17
2.5 Related Work 20
2.6 Conclusion and Future Work 21

3 ABLE: Using Adversarial Pairs to Construct Local Models for Explaining Model Predictions **23**
3.1 Introduction 24
3.2 Related Work 26
3.3 Background 27
3.3.1 LIME-Based Explanations 27

3.3.2 Adversarial Data Points . . . 28
3.4 Motivation . . . 28
3.5 Approach . . . 29
3.5.1 Problem Setup and Notation . . . 30
3.5.2 Overview of Our Approach . . . 30
3.5.3 Step 1: Neighborhood Generation . . . 30
3.5.4 Step 2: Adversarial Pair Generation . . . 31
3.5.5 Step 3: Local Model Training . . . 31
3.5.6 Multi-Class Extension . . . 31
3.6 Experiment Design . . . 32
3.6.1 Datasets . . . 32
3.6.2 Target Models . . . 33
3.6.3 Adversarial Sample Generation . . . 34
3.6.4 Hyperparameters . . . 34
3.6.5 Baselines . . . 34
3.6.6 Evaluation Metrics . . . 36
3.6.7 Experimental Procedure . . . 36
3.7 Results and Discussion . . . 37
3.7.1 Hyperparameter Analysis . . . 37
3.7.2 Fidelity Comparison . . . 38
3.7.3 Stability Comparison . . . 38
3.7.4 Runtime Analysis . . . 38
3.7.5 Discussion . . . 39
3.8 Conclusion and Future Works . . . 39

4 DD-CAM: Minimal Sufficient Explanations for Vision Models Using Delta Debugging 41
4.1 Introduction . . . 41
4.2 Related Work . . . 43
4.2.1 Model-Agnostic Explanations . . . 43
4.2.2 Sensitivity-Based Explanations . . . 43
4.2.3 Class Activation Mapping Explanations . . . 44
4.3 Background . . . 44
4.3.1 Vision Model Representations . . . 44
4.3.2 Delta Debugging . . . 45
4.4 Motivation . . . 45
4.5 Approach . . . 45
4.5.1 Step 1: Activation Extraction . . . 46
4.5.2 Step 2: Subset Selection via Delta Debugging . . . 46
4.5.3 Step 3: Saliency Map Generation . . . 48

4.6 Experimental Design . . . 49
4.6.1 Datasets . . . 50
4.6.2 Models . . . 50
4.6.3 Baselines . . . 51
4.6.4 Evaluation Metrics . . . 51
4.7 Results and Discussion . . . 51
4.7.1 Faithfulness Evaluation (RQ1) . . . 52
4.7.2 Localization (RQ2) . . . 52
4.8 Limitations . . . 52
4.9 Conclusion . . . 53
4.10 Appendix: Proof of Proposition 1 . . . 53

5 Explanation-Guided Model Repair 57
5.1 Introduction . . . 57
5.2 Problem Setup . . . 57
5.2.1 Key Insight . . . 58
5.3 Method . . . 60
5.3.1 Stage 1: Concept Bank Construction . . . 60
5.3.2 Stage 2: Failure Pattern Discovery . . . 61
5.3.3 Stage 3: Failure Localization and Two-Gate Validation . . . 61
5.3.4 Stage 4: Targeted Synthetic Augmentation . . . 62
5.3.5 Scope . . . 63
5.4 Preliminary Results . . . 63
5.4.1 Dataset . . . 63
5.4.2 Baselines . . . 63
5.4.3 Results . . . 64
5.5 Remaining Tasks . . . 64
5.6 Conclusion . . . 65

6 Conclusion and Timeline 66
6.1 Summary of Contributions . . . 66
6.2 Timeline . . . 66

Chapter 1

Introduction

1.1 Research Overview

Machine learning (ML) models, particularly deep neural networks, have achieved remarkable performance across domains including healthcare, finance, and autonomous systems [66]. Despite their predictive power, these models remain largely opaque; they do not transparently reveal their internal decision-making processes [69, 99, 11]. This lack of interpretability creates challenges in understanding, testing, and debugging model behavior, which affects user trust and raises concerns about accountability, reliability, and fairness in high-stakes applications.

Explainable Artificial Intelligence (XAI) has emerged as a critical research area to address this opacity [11, 68]. XAI focuses on developing tools and methods that make ML models' decision processes interpretable. While existing explanation methods, ranging from local surrogate models like LIME [95] to gradient-based attribution methods like Grad-CAM [104], have advanced our ability to inspect model behavior, they share critical limitations. Local surrogate methods suffer from instability and poor fidelity [121, 8]. CAM-based methods aggregate all representational units, producing cluttered saliency maps. Most fundamentally, existing explanation methods are largely descriptive: they tell us what a model attends to, but do not provide a direct pathway to fixing model failures.

This dissertation addresses the interpretability challenge through a progression of contributions:

- **Stage 1 (Understand):** How many features determine a prediction? We show that most predictions depend on only a few features (Chapter 2).
- **Stage 2 (Explain, Tabular):** We build faithful local explanations by targeting the decision boundary through adversarial pair generation (Chapter 3).
- **Stage 3 (Explain, Vision):** We adapt delta debugging [136] to identify 1-minimal sufficient subsets for vision models (Chapter 4).
- **Stage 4 (Repair):** We identify recurring failure-driving concepts and repair them through

targeted synthetic augmentation that fills under-represented cells of the joint concept-class distribution, without group labels (Chapter 5).

1.2 Summary of Publications

Chapters 2–5 each correspond to a research paper. Chapter 2 is published at IWCT 2024. Chapter 3 is published at KDD 2026. Chapter 4 targets NeurIPS 2026. Chapter 5 targets AAAI 2027.

Additional publications completed during the doctoral program include: S. Shree, K. Khadka, Y. Lei, R. N. Kacker, D. R. Kuhn, "Constructing Surrogate Models in Machine Learning Using Combinatorial Testing and Active Learning," ASE 2024; S. N. Pereira, K. Khadka, Y. Lei, "TabKD: Tabular Knowledge Distillation through Interaction Diversity of Learned Feature Bins," arXiv, 2026; K. Khadka et al. "Synthetic Data Generation Using Combinatorial Testing and Variational Autoencoder," ICSTW 2023 [58]; K. Khadka et al. "A Combinatorial Approach to Hyperparameter Optimization," CAIN 2024.

Chapter 2

Assessing the Degree of Feature Interactions that Determine a Model Prediction

This chapter is based on: K. Khadka, S. Shree, Y. Lei, R. N. Kacker, D. R. Kuhn, "Assessing the Degree of Feature Interactions that Determine a Model Prediction," IWCT 2024.

Abstract

Machine Learning (ML) models rely on capturing important feature interactions to generate predictions. This study is focused on validating the hypothesis that model predictions often depend on interactions involving only a few features. This hypothesis is inspired by t-way combinatorial testing for software systems. In our study, we utilize the notion of Shapley Additive Explanations (SHAP) values to quantify each feature's contribution to model prediction. We then use a greedy approach to identify a minimal subset of features (t) required to determine a model prediction. Our empirical evaluation is performed on three datasets: Adult Income, Mushroom, and Breast Cancer, and three classification models: Logistic Regression, XGBoost, and SVM. Through our experiments, we find that the majority of predictions are determined by interactions involving only a subset of features.

Keywords: Feature Interaction, Model Prediction, Combinatorial Testing

2.1 Introduction

ML model predictions are made by recognizing interactions among various features. It is important to understand how models' predictions are made in high-stake fields such as healthcare [7], finance [61], and autonomous driving [29]. Understanding the impact of feature interactions on model predictions are vital to analyze, interpret and debug the prediction process of a model. The

identification of a minimal subset of features for each model's predictions help us to both interpret and fix issues with the model's predictions.

Many studies have been conducted to study feature importance in ML models, shedding light on the influence of individual features on model predictions. Feature importance can be categorized into two types: model-level and instance-level. Model-level methods, such as Gini importance [85], permutation importance [20], as well as global surrogate models like linear models [34], and decision trees [91], provide insights into a feature's overall impact on the model decision process for all possible instances . These methods help identify important features and potential biases at the model level but may not explain how those features affect individual predictions. Instance-level methods, like Local Interpretable Model-agnostic Explanations (LIME) [96] and Shapley Additive Explanations (SHAP) [75], focus on explaining individual model decisions. LIME utilizes a simpler, local model to explain the rationale behind the decision process for an individual instance, while SHAP measures the contribution of individual features to model predictions for a specific instance.

While previous works mostly focus on quantifying feature importance, they do not explicitly identify a minimum subset of features that determines a model's prediction. In our study, we hypothesize that the model prediction is often determined by the interaction of only a few features, i.e. t out of n total features, where t is typically a small number, while the features that are important for different predictions may be different. This hypothesis is a motivation from t-way combinatorial testing in software testing, where system failures can be identified with t feature interaction, where t is typically a small number [63]. Validating this hypothesis will give confidence and thus enable future work in applying t-way testing in the domain of machine learning such as model interpretation, testing and debugging.

To empirically validate our hypothesis, we develop an approach to identify a minimal subset of features whose interactions are important for preserving a model prediction. In our approach we use SHAP values [75] to measure the contribution of individual features and their interactions. SHAP values quantify the contribution of each feature to model predictions at an instance-level. SHAP values are additive, meaning that their sum captures the total effect of feature interactions on the prediction [108]. Each SHAP value for a feature inherently considers all the possible interactions with other features. Moreover, features can have either positive or negative influences on the model's classification decision, directing it towards class 1 or class 0, respectively.

To identify a minimal feature subset, we begin with an empty set R and iteratively add features based on their SHAP values. This process includes supporting and opposing features—those that positively or negatively affect the model's classification, respectively. The addition of supporting features to R continues until the cumulative SHAP values in R are sufficient to maintain the model's prediction (e.g., maintains the same class prediction or keeps the prediction probability above a certain level), despite potential negative contributions from the opposing features. We emphasize that we can use the cumulative SHAP values to measure the total contribution of the features in R because SHAP values are additive, i.e., they do not only capture the contributions made by individual features but also the contributions due to feature interactions.

We conducted experiments on three datasets, Adult Income [13], Mushroom [3], and Breast Cancer [130], using three classification models, Logistic Regression [34], XGBoost [30], and Support Vector Machine [51]. Our results provide strong support for our hypothesis that only a small subset of features are required to preserve model predictions. We observed that the majority of instances achieved accurate predictions with fewer than five interacting features. On average, two or three features were enough to maintain accurate predictions. To verify that the subset of features we identified using our approach are truly determining a model prediction, we retrained the model using the features we identified and then checked if the retrained model produces the same prediction.

The structure of this paper is organized as follows: Section II provides the foundational background. Section III presents our approach to identification of feature subset. Section IV describes our experimental design and presents the results. In section V, we review the related studies. Finally, Section VI concludes the paper, summarizing our findings and suggesting future research directions.

2.2 Background

2.2.1 Machine Learning Tasks

Machine Learning models are algorithms that generally perform two key tasks: regression and classification. The models are trained with input features and corresponding outcomes, allowing them to learn the relationships between them. Once trained, these models can be used for performing inference tasks. The regression model predicts continuous outcomes, e.g., predicting real estate prices [79] or forecasting weather [53]. On the other hand, classification model predicts discrete labels, e.g., credit card fraud detection [44], spam email detection [106], or diagnosing medical diseases [111].

2.2.2 Decision Making in Classification Task

Consider a dataset D with N instances, where $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, where x is the input feature vector, and y is the class label. The dataset D is split into a training set and a test set. M is a classification model, which maps input features to class labels: $M : x \rightarrow y$, parameterized by weight θ. Classification task consist of the following two phases:

Training Phase

During the training phase, M learns to map input features x to their corresponding class labels y. The model M learns by updating model parameters θ during training. The process begins by defining a loss function $\mathcal{L}\left(\left(\left(x_i; \theta\right), y_i\right), \hat{y}_i\right)$, which measures the error between the model's predictions $\hat{y}_i$ and the actual labels y_i. The objective is to minimize the loss function $\mathcal{L}$. The model parameters are iteratively updated using an optimization algorithm such as gradient descent to minimize the total loss on the training dataset.

Inference Phase

After the training phase, model M is applied to unseen test data. The model M utilizes its learned parameters θ to make predictions on test instances. The model is said to be an effective model if it can generalize well on the test set. The performance of the model is evaluated using different metrics such as accuracy, precision, recall, and F1 scores.

Feature Contribution for Decision Making: A model's decision is the contribution of individual features and their interactions. Different features may have varying levels of influence on the model's predictions. Some features are determining features, while others may have minimal or no impact. This requires identifying the important features and quantifying their contributions for the final decision. Understanding how different features influence the model prediction helps interpret the model prediction.

2.2.3 SHAP Values

SHAP values are developed based on game theory, and are used to quantify the contribution of each feature to a model prediction. Each feature is treated as a "player" in a game and the prediction is the total "payout" [108]. SHAP values considers all possible combinations of features and ensure a fair distribution of the prediction among all features based on their individual contributions.

These values can be positive or negative, indicating the direction of influence a feature has on the model's prediction. In binary classification, a positive SHAP value adds to the final prediction, indicating a push towards the positive class, while a negative value subtracts from the final prediction, indicating a push towards the negative class. This is important for interpreting the model's behavior due to different feature contributions. In addition, SHAP values provide local interpretability allowing the examination of feature contribution on an individual level.

Calculating SHAP Value

SHAP Value is calculated with the formula:

$$\phi_j = \sum_{S \subseteq \{1,\ldots,p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} \left(v(S \cup \{j\}) - v(S)\right) \tag{2.1}$$

Here, ϕ_j is the SHAP value for feature j. S is a subset of features excluding j, p is the total number of features, and $v(S)$ is the prediction value for the feature set S. This equation distributes the prediction output fairly among all contributing features based on their individual impact, which is the core principle of SHAP value.

Explaining Predictions Using SHAP Values

To explain a model prediction with SHAP values involves combining the base value with the SHAP values associated with each feature for the prediction. The base value is the average prediction of the model over the training set, serving as a starting point. SHAP values are then added to

Table 2.1: Summary of Datasets Used in the Study

Dataset	Total Instances	Class 0 Instances	Class 1 Instances	Training Instances	Test Instances	Categorical Features	Continuous Features	Classes
Adult Income	48842	37155	11687	39074	9768	8	6	2
Mushroom	8124	3916	4208	6499	1625	22	0	2
Breast Cancer	569	357	212	455	114	0	30	2

the base value (BV), which either increases or decreases the final prediction based on the feature's contribution towards the prediction. Mathematically, a final prediction (FP) can be expressed as.

$$\text{FP} = \text{BV} + \sum_{i=1}^{N} \phi_i \tag{2.2}$$

where N is the number of features. The sum of base value and cumulative sum of SHAP values is equal to the final prediction. In binary classification, the final prediction is transformed into a prediction probability through a logistic function. The prediction probability is assigned a class label based on a classification threshold. A common threshold for binary classification is 0.5, which means if the probability is greater than 0.5 then the assigned class is 1; otherwise it is class 0.

Additive Nature of SHAP Values

The additive nature of SHAP values refers to the fact that the amount of contribution for a feature set is equal to the sum of the amounts of contribution of individual features in the set. In another word, SHAP values inherently captures the effects of feature interactions by considering all possible combinations of features during computation. These SHAP values can be added to the base value to obtain a final prediction.

For instance, Figure 1, presents a waterfall plot displaying SHAP values for an instance in Adult Income Dataset. In this context, the base value is -1.777. To calculate the final prediction for the instance, we sum up the individual SHAP values with the base value, resulting in a sum of -2.16. This summation illustrates the additive nature of SHAP values, showing how they are added together to produce the final prediction. Subsequently, a logistic function is applied to the final prediction to derive the class label.

2.3 Approach

This section presents our approach that uses SHAP values to empirically validate the hypothesis that only a subset of features are sufficient for a model prediction in a binary classification task. We aim to identify the minimal subset of features required for model predictions, and we provide a detailed description of our approach in Algorithm 1.

Algorithm 1 Feature Subset Identification Using SHAP Values

Input: Dataset D, Machine Learning Model M, Classification Threshold T

Output: List of minimum subsets of features required for each test instance to preserve the model prediction.

```
Split D into training and test sets.
Train M on the training set.
Compute Base Prediction Value, BV, using M and the training set.
Initialize a list All_R to store subsets for all test instances.
for each instance in the test set do
    Initialize the instance-specific subset R as empty.
    Compute SHAP values, φ, for all the features.
    Predict the class label for the instance using model M.
    if Class label is 1 then
        Identify the supporting set of features, S, i.e., features with positive SHAP values.
        Sort S in descending order based on their SHAP values.
        Identify the opposing set of features, O, i.e., features with negative SHAP values.
    else
        Identify the supporting set of features, S, i.e., features with negative SHAP values.
        Sort S in descending order based on the absolute values of their SHAP values.
        Identify the opposing set of features, O, i.e., features with positive SHAP values.
    end if
    Calculate Σ_{i=1}^{n} |φ_{O_i}|, where n is the number of features in O.
    for t = 1 to the number of features in S do
        Append feature S_t to R.
        Compute Margin as BV + Σ_{i=1}^{t} φ_{S_i} − T.
        if |Margin| > Σ_{i=1}^{n} |φ_{O_i}| then
            Break the loop, R now contains a sufficient subset.
        end if
    end for
    Append the subset R for this instance to All_R.
end for
Return All_R
```

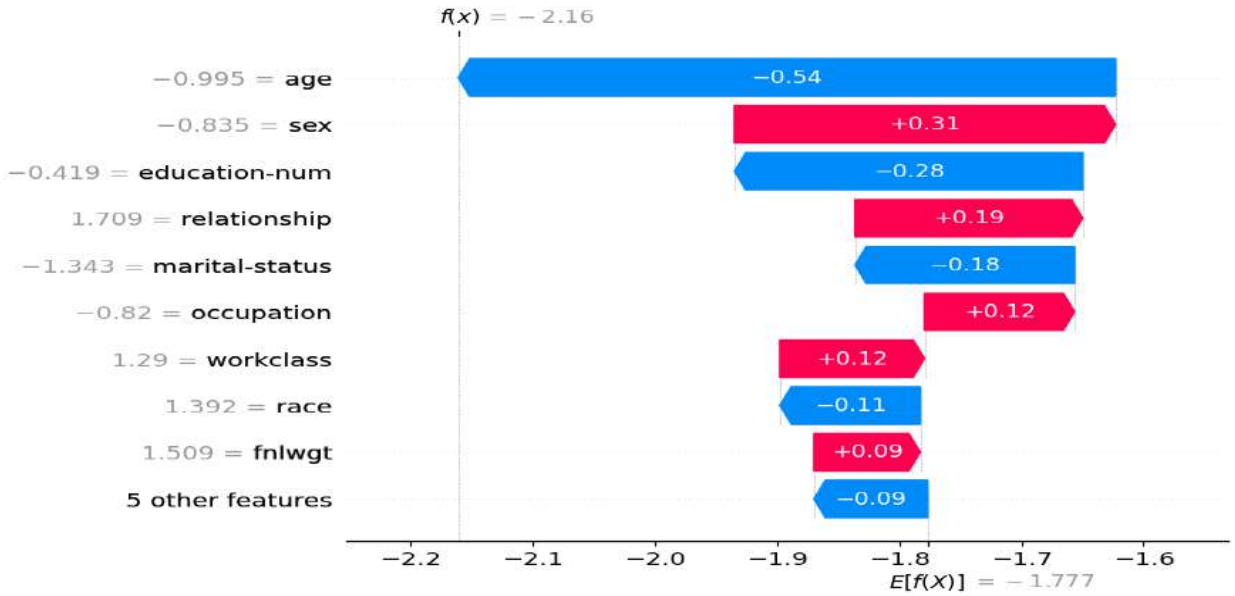

Figure 2.1: Waterfall Plot Illustrating SHAP Values For a Class 0 Standardized Sample of Adult Income Predicted by Logistic Regression. The Blue and Red Bars Represent SHAP Values, with Positive Values Favoring Class 1 and Negative Values Favoring Class 0, respectively. Normalized Feature Values Are Displayed to the Left of Respective Feature Names.

2.3.1 Step 1: Compute and Sort SHAP Values

Computation of SHAP Values (Lines 1-7 of Algorithm 1)

We begin by training model M with a training dataset. Next we compute SHAP values, ϕ, for each instance in the test set. SHAP values can be computed using existing SHAP explainers, e.g., shap.Explainer, shap.TreeExplainer, or shap.KernelExplainer [75]. For our purpose, we use the most commonly used shap.Explainer.

Sorting SHAP Values Based on Predicted Class Label (Lines 8-17 of Algorithm 1)

Given the model M, we predict a class label for the test instance under consideration. We then identify the SHAP values based to their contribution towards this prediction. For instances classified as 1, we select features with positive SHAP values, and sort them in a non-increasing order by their magnitude. Likewise, for instances labeled as 0, we choose features with negative SHAP values and sort them similarly in non-increasing order based on their absolute values. These sorted features are supporting features (S) for the prediction, while the remaining features are considered as opposing features (O). For class 1, the opposing features contain with negative SHAP values, whereas for class 0, they contain features with positive SHAP values.

2.3.2 Step 2: Margin-based Iterative Top t Feature Identification (Lines 18-28 of Algorithm 1)

A minimal set of features sufficient for preserving the model prediction is a subset of features from the supporting set. This subset is determined iteratively by adding features from the supporting set and recalculating the margin after each addition. The margin M is defined as the difference between the Base Prediction Value (BV) using model M and the Classification Threshold T, adjusted by the cumulative SHAP values of the added features. The objective is to find the smallest number of features whose absolute margin exceeds the sum of the absolute SHAP values of the opposing features. We use margin to determine whether the contribution of the top t features, based on their SHAP values, is sufficient to maintain the original classification of a test instance.

Margin Calculation (Line 14 of Algorithm 1)

The margin is calculated by the equation:

$$\text{Margin} = \text{BV} + \sum_{i=1}^{t} \phi_i - \text{T} \tag{2.3}$$

where BV is the base value, which is the average prediction made by the model across all instances. The expression $\sum_{i=1}^{t} \phi_i$ represents the aggregate sum of the SHAP values for the top t features. The term T is classification threshold.

Iterative Identification of Top t Features (Lines 15-21 of Algorithm 1)

We begin with $t = 1$ and calculate margin using the SHAP value(s) of the top t features. We then compare the absolute margin against the cumulative sum of absolute SHAP values of all the opposing-direction features.

If the initial set of t features is insufficient to maintain the classification, i.e. absolute margin is less then the sum of absolute value of the opposing SHAP values, we increment t to include an additional feature in the margin calculation. We then repeat the evaluation process for identifying the top t features as described above. This iterative process continues until we find a minimum number of t features whose cumulative contribution, including the base prediction, preserves the original prediction. Note that this process must terminate since the cumulative contribution of subset of supporting features must match the original prediction.

Through this systematic approach, we leverage SHAP values to identify top t feature set sufficient for preserving model predictions.

2.3.3 Example

Here, we explain a working example with Figure 1, which displays a waterfall plot of SHAP values for a standardized instance from the Adult Income dataset trained with LR. The horizontal axis

represents the log-odds values. The base value log-odds for is BV = -1.777, the log-odds classification threshold is 0, and the log-odds value of the final prediction is -2.16. After applying logistic function, the model's predicted class label for this particular example is 0.

Our initial step starts with sorting the features with negative SHAP values as the class label is 0. The sorted features comprise [age, education-num, marital-status, race, and 5 other features]. For $t = 1$, we calculate the margin using the 'age' feature's SHAP value, utilizing Equation 3. The resulting margin is calculated as $Margin = -2.317$.

Next, we calculate the sum of all opposing SHAP values, which is 0.89. To check the classification's preservation, we compare the absolute value of the margin to the sum of opposing SHAP values. Here, the absolute value of the margin, 2.317, is greater than the sum of opposing SHAP values, 0.89, confirming the preservation of the classification. Thus, for this instance, $t = 1$ preserves the classification.

2.4 Experiment and Results

We aim to validate our hypothesis by answering the following research question:

- How many features determine the prediction of an instance?

2.4.1 Datasets:

In our experiment, we choose three commonly used real-world datasets for binary classification tasks. Each dataset consists of categorical or continuous features or both types of features. We also have balanced and imbalanced datasets. These variations allow us to test our hypothesis across different scenarios. The datasets include:

- **Adult Income** – The dataset is used to predict whether an individual income exceeds more than $50,000 per year [13]. It represents an imbalanced dataset with uneven class distribution and includes both categorical and continuous features. It has been widely used in binary classification tasks.

- **Mushroom Dataset** - The Mushroom dataset is used for classification of mushrooms as either edible or poisonous, based on 23 species within the Agaricus and Lepiota families [3]. It consists of categorical features describing physical attributes of mushrooms. It is a balanced dataset, with a nearly equal distribution of classes.

- **Breast Cancer** - The Breast Cancer dataset is for the classification of breast cancer tumors into malignant or benign categories [130]. It consists of 569 instances described by 30 continuous features each. This dataset is utilized for health and medicine research, specifically in tasks related to classification.

Table 2.2: Statistical Summary of t Values

Datasets	Models	Mean	Median	Maximum	Minimum	Total Features
Adult Income	**LR**	1.32	1	10	1	14
	XGBoost	1.62	1	11	1	
	SVM	1.17	1	10	1	
Mushroom	**LR**	2.76	2	11	1	22
	XGBoost	1.01	1	3	1	
	SVM	2.96	2	14	1	
Breast Cancer	**LR**	1.6	1	7	1	30
	XGBoost	1.82	2	7	1	
	SVM	4.05	4	20	1	

For experimental preparation, we perform 80:20 split for training and testing purpose. Data preprocessing includes standard normalization. Table 1 provides a summary overview of these datasets.

2.4.2 Classification Models:

We use three types of classification models. They are commonly used for tabular data. We train each of the three model types with each of the three datasets and thus obtain a total of nine models:

- **Logistic Regression (LR)** – LR is a simple model in machine learning for classification task [34]. Its linear nature is useful in scenarios where relationship between features and outcomes are expected to be linear.
- **XGBoost** – XGBoost is an implementation of gradient boosted trees [30]. It has high utility in handling large and complex dataset. In addition, it also handles non-linear interactions between features and thus helps understand complex non-linear feature interactions.
- **Support Vector Machine (SVM)** – SVM is a kernel based classification model that uses support vectors to identify an optimal hyperplane to maximize the margin between the classes [130]. SVM is suited for high dimensional features.

The models are trained using scikit-learn library [89] on the training set of each dataset, using standard hyper-parameter values.

2.4.3 SHAP Value Computation

SHAP values are computed using the SHAP library [75]. In particular, we use shap.Explainer from the SHAP library.

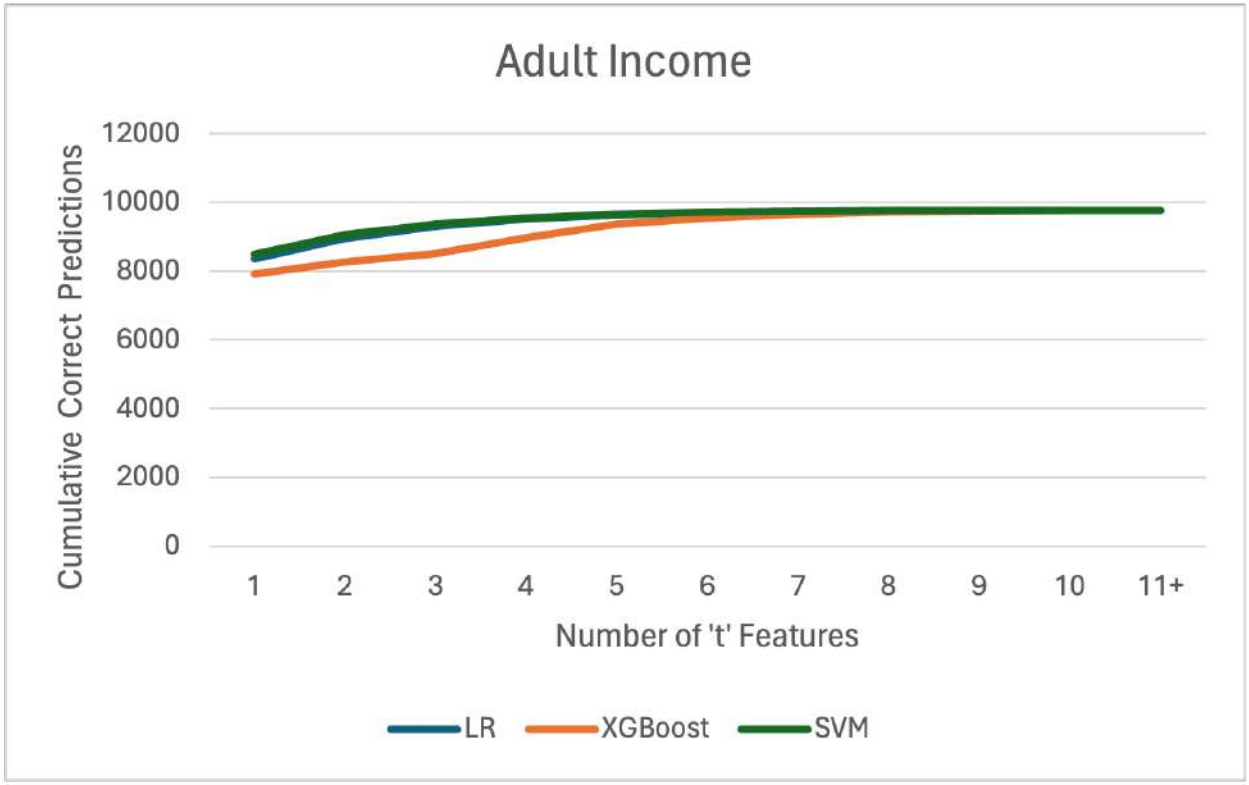

Figure 2.2: Adult Income Dataset: Cumulative Predictions Determined by t Features

2.4.4 Results and Discussion

We begin our results and discussion with an instance-level examination, followed by an aggregated dataset-level analysis.

Instance-Level Analysis

We identify the number of features that are sufficient to maintain the original prediction for each test instance. Figure 2 presents the results for the Adult Income dataset. As we increase t from $t = 1$, we can see the cumulative number of correct predictions increasing for all models. However, beyond $t = 6$, the graph levels off. This suggests that while additional features initially were required for some instances, there comes a point where further additions are not needed. This leveling off is indicative of the diminishing benefit of adding more features. For all models, more than 90% of the instances achieved correct predictions using fewer than 5 interacting features identified using our approach.

Figure 3 shows the results for the Mushroom dataset. Here, we again initiate t at 1 for each instance. For LR and SVM, the cumulative number of correct predictions gradually increases as we include more features, reaching a plateau after $t = 7$. This means that additional features beyond this point are not needed to preserve correct predictions. In contrast, we observe a striking trend with XGBoost. The majority of instances attained correct predictions with just $t = 2$ features.

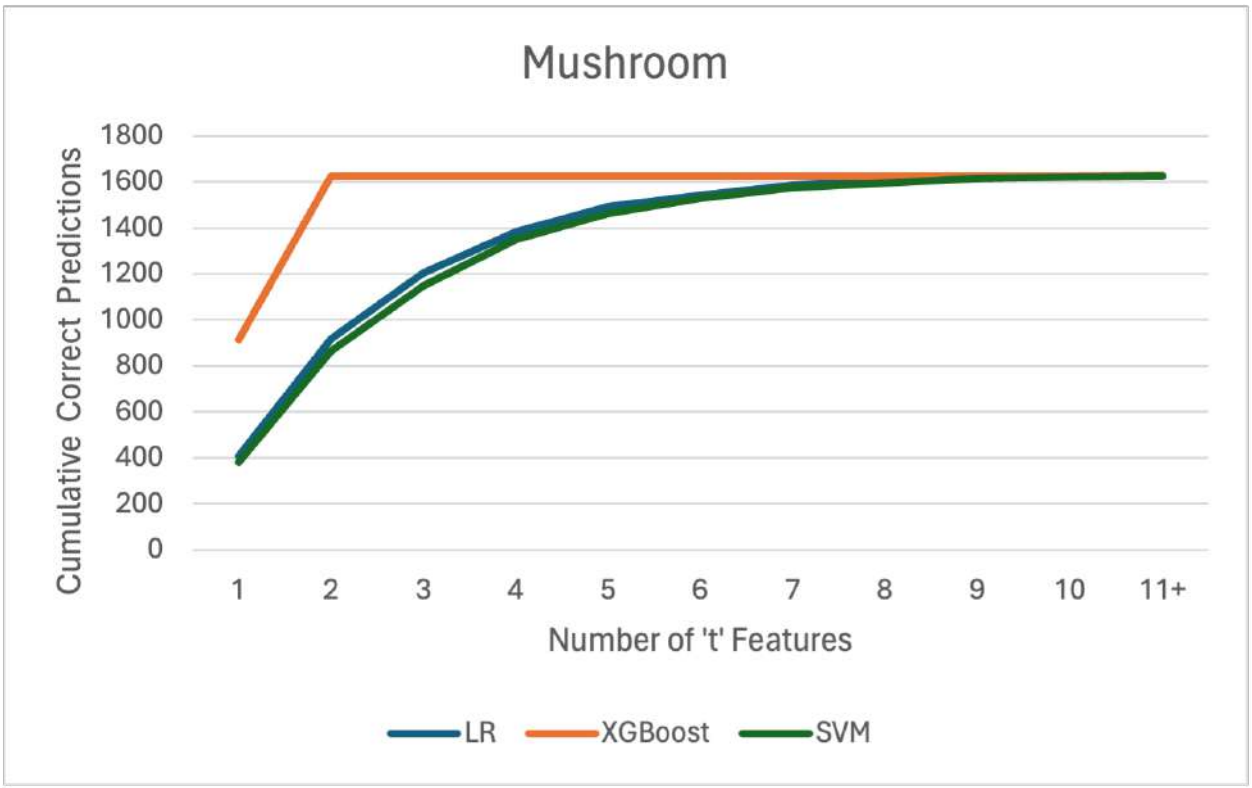

Figure 2.3: Mushroom Dataset: Cumulative Predictions Determined by t Features

This implies that, two features sufficed to achieve accurate predictions for nearly all instances. This can be attributed to XGBoost's strong feature selection capabilities.

Figure 4 presents the results for the Breast Cancer dataset. We start with $t = 1$ for each instance. For all the three models, the cumulative number of correct predictions increases as we increase more features. The curves reach a plateau after $t = 7$ which suggests that at most seven features are enough to get most predictions correct.

Many machine learning models may contain redundant or irrelevant features that do not contribute meaningfully to predictions. Our results demonstrate that with just a small subset of features, predictions can be preserved. It's worth noting that while some instances may require the same number of features to maintain predictions, the specific features that are needed may vary.

Dataset-Level Analysis

The dataset-level analysis provides insights across various datasets and models. For each model and the corresponding test set, we calculated four key statistical properties: mean value, median value, maximum value, and minimum value of t. The numbers are shown in Table II.

Mean Value: For the Adult Income dataset, all three models had small mean values. It indicates that, on average, only a small subset of features was needed to preserve predictions. SVM had a mean t of 1.17; LR had a mean t of 1.32; while XGBoost had a slightly higher mean t of

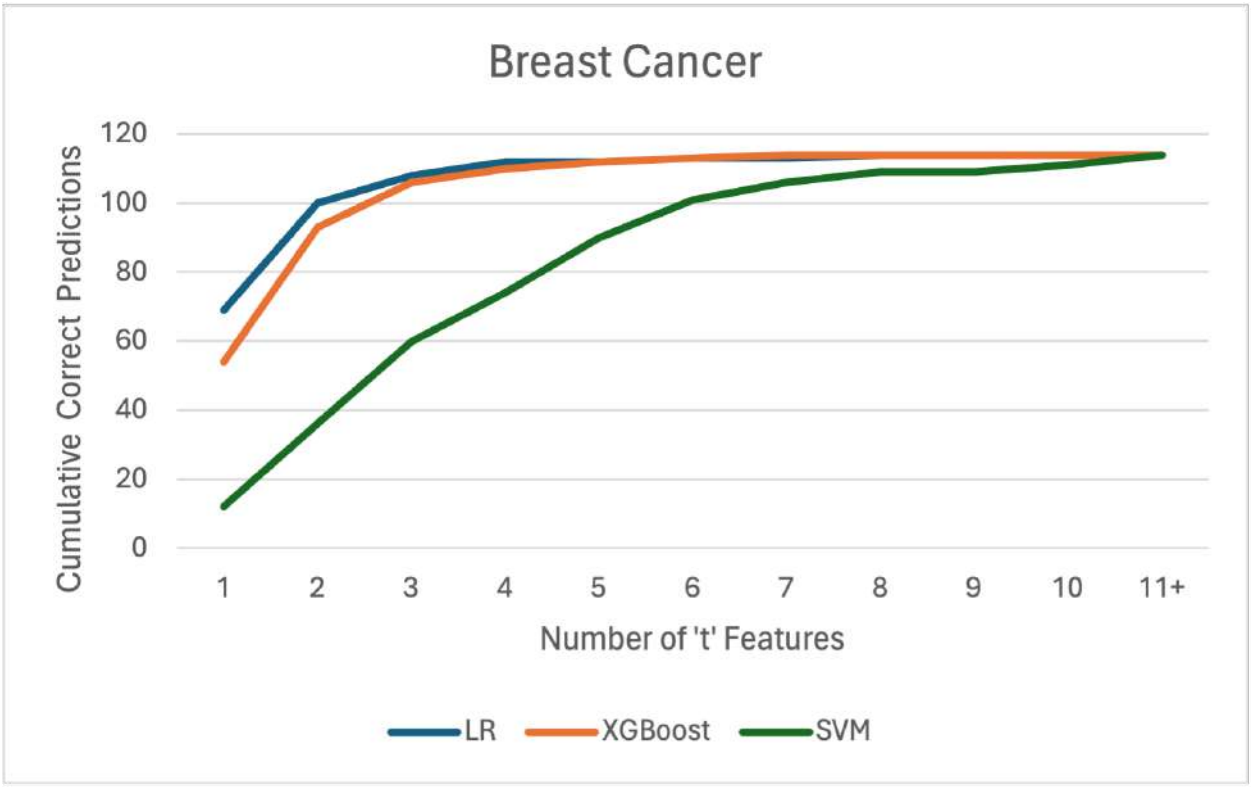

Figure 2.4: Breast Cancer Dataset: Cumulative Predictions Determined by t Features

1.62. For the Mushroom dataset, the mean value ranges from 1.01 to 2.96. For the Breast Cancer dataset, the mean value ranges from 1.6 to 4.05. Overall, a maximum average of 4 features are sufficient for preserving model predictions across the datasets.

Median Value: In both the Adult Income and Mushroom datasets, the median t value for the majority of instances is only one or two features needed to maintain accurate predictions. For breast cancer, LR has a median of 1, XGBoost has a median of 2, and SVM has a higher median value of 4.

Maximum Value:. In case of the Adult Income dataset, LR and SVM had a maximum of 10 features for a single instance, while XGBoost needed up to 11 features. Similarly, for the Mushroom dataset, LR required a maximum of 11 features, SVM a maximum of 14 features, while XGBoost needed a maximum of only 3 features. For Breast cancer, LR and XGBoost had a maximum of 7 features, while SVM had a maxium of 20 features. It is interesting to note that these maximum values are significantly lower than the total numbers of features and instances requiring these maximum number of features are very few.

Minimum Value:. For all instances across all datasets, the minimum t was consistently 1, indicating that there were instances where only a single feature was sufficient to preserve the original prediction.

In summary, our dataset-level analysis shows that, on average, a small number of features are needed to maintain accurate predictions across various instances.

Table 2.3: Model Performance Comparison Based on Randomly Picked Samples

Datasets	**Models**	**1**		**2**		**3**		**4**		**5**		**Model Accuracy**
		t	**Acc**	**t**	**Acc**	**t**	**Acc**	**t**	**Acc**	**t**	**Acc**	
Adult Income	**LR**	1	0.76	1	0.77	3	**0.79**	1	0.76	2	0.78	0.82
	XGBoost	4	**0.82**	5	0.81	1	0.76	1	0.81	1	0.81	0.85
	SVM	1	**0.8**	5	0.78	2	0.76	1	0.79	1	0.8	0.81
Mushroom	**LR**	1	0.9	4	**0.94**	5	0.87	3	0.91	4	0.81	0.93
	XGBoost	1	**0.98**	1	0.98	1	0.98	1	0.98	1	0.98	0.98
	SVM	2	0.89	4	**0.94**	4	0.93	2	0.89	3	0.9	0.92
Breast Cancer	**LR**	3	0.9	2	**0.97**	2	0.89	1	0.88	2	0.9	0.97
	XGBoost	1	0.84	2	**0.95**	1	0.82	2	0.93	3	0.92	0.95
	SVM	4	0.82	6	**0.98**	4	0.93	3	0.86	2	0.84	0.97

Validation of Model Predictions with Reduced Feature Set

To validate that the subset of features we identified using our approach truly determine model predictions, we retrained the model using only the identified features and subsequently examined whether the retrained model produced the same predictions as the original. For this evaluation, we randomly selected five instances from the test set for each combination of dataset and model. The original model was retrained using the features identified as most important for these selected instances. These newly trained models, which used the reduced feature set, were tested on the test set, and their accuracy was compared to models trained with the entire feature set. All the models trained on reduced feature subset have an accuracy within 6% of the original model accuracy. For some instances, such as instance 2 in Mushroom dataset trained on LR, achieved even higher accuracy than the original accuracy with $t = 4$. This observation suggests that utilizing only a few features can sometimes produce better results compared to using all features. Detailed results are presented in Table III.

2.5 Related Work

Several studies have explored the feature importance in the context of model predictions [93][39][80]. Feature importance study can be split into two parts: Model-level and instance-level approaches.

Model-level approaches provide feature importance at the entire dataset level, including feature interaction techniques such as Gini importance [85], permutation importance [20], and Principal Component Analysis (PCA) [131]. Gini importance is based on entropy that measures a feature's contribution by checking how often a feature is used to split the data, whereas permutation importance measures the decrease in a model's accuracy when the feature values are randomly shuffled. Both approaches rank features based on their contribution to model predictions. Principal Com-

ponent Analysis (PCA) is a statistical analysis for feature reduction, identifying a few features out of the total feature set at the dataset level [131][114].

Unlike model-level approaches, our approach works at the instance level. In particular, we try to identify a set of features that are sufficient to preserve an individual prediction. As observed in our experiments, the feature sets that preserve individual predictions differ from instance to instance.

Instance-level approaches, e.g. LIME [96] and SHAP [67], provide feature importance at the instance level. LIME constructs an interpretable surrogate model around individual predictions to approximate the original model predictions. These surrogate models closely mimic the behavior of original models, explaining the contributions of individual features to a particular instance. SHAP values are developed based on game theory. They capture the contributions of individual features towards a model prediction by considering all possible combinations of features [67][72].

Instance-level approaches are developed to explain individual model decisions. They do not identify a minimum set of features for preserving predictions. Note that our approach uses SHAP values to measure the contributions of individual features. In this respect, our approach is complementary to approaches that measures feature importance at the instance level.

Our work is inspired by t-way combinatorial testing, a software testing approach to analyze the effects of interactions among t parameters on a system's output [21]. In particular, our work is inspired by a similar study that investigates the degree of parameter interactions for software failures [63]. T-way testing has also been used in different machine learning tasks for detecting model biases [87], generating synthetic data [58], and testing deep neural networks [24], etc. We hope that our current work provides additional support for the effectiveness of applying t-way testing to machine learning systems.

2.6 Conclusion and Future Work

We developed an approach that uses SHAP values to identify a minimal subset of features that preserves a model prediction. We conducted an empirical study to assess the degree of feature interactions that determine individual model predictions, using three datasets, Adult Income, Mushroom and Breast cancer and three classification models, Logistic Regression, XGBoost, and SVM. Our experimental results demonstrate that model predictions could be preserved using fewer than five features for the majority of instances. Furthermore, we verified the identified reduced feature sets by retraining the models using only the features we identified and subsequently checked whether the retrained models produced consistent predictions with the original ones.

In the future, we aim to broaden the scope of our research by considering a variety of machine learning models and diverse datasets to gain a more comprehensive understanding of the underlying dynamics. We also plan to explore feature subset in multi-class classification tasks. Our long term goal is to pave the way for the application of combinatorial testing in the field of machine learning.

Acknowledgement

This work is supported by a research grant (70NANB21H092) from Information Technology Lab of National Institute of Standards and Technology (NIST).

Disclaimer: Certain equipment, instruments, software, or materials are identified in this paper in order to specify the experimental procedure adequately. Such identification is not intended to imply recommendation or endorsement of any product or service by NIST, nor is it intended to imply that the materials or equipment identified are necessarily the best available for the purpose.

Chapter 3

ABLE: Using Adversarial Pairs to Construct Local Models for Explaining Model Predictions

This chapter is based on: K. Khadka, S. Shree, P. Budhathoki, Y. Lei, R. N. Kacker, D. R. Kuhn, "ABLE: Using Adversarial Pairs to Construct Local Models for Explaining Model Predictions," KDD 2026.

Abstract

Machine learning models are increasingly used in critical applications but are mostly "black boxes" due to their lack of transparency. Local explanation approaches, such as LIME, address this issue by approximating the behavior of complex models near a test instance using simple, interpretable models. However, these approaches often suffer from instability and poor local fidelity. In this paper, we propose a novel approach called Adversarially Bracketed Local Explanation (ABLE) to address these limitations. Our approach first generates a set of neighborhood points near the test instance, x_{test} , by adding bounded Gaussian noise. For each neighborhood point D, we apply an adversarial attack to generate an adversarial point A with minimal perturbation that results in a different label than D. A second adversarial attack is then performed on A to generate a point A' that has the same label as D (and thus different than A). The points A and A' form an adversarial pair that brackets the local decision boundary for x_{test} . We then train a linear model on these adversarial pairs to approximate the local decision boundary. Experimental results on six UCI benchmark datasets across three deep neural network architectures demonstrate that our approach achieves higher stability and fidelity than the state-of-the-art.

Keywords: Explainable AI, Interpretability, Adversarial Learning, Local Explanations, Local Models, Decision Boundaries.

3.1 Introduction

Modern machine learning (ML) models such as deep neural networks have become increasingly accurate, finding wide adoption in real-world applications ranging from healthcare diagnostics to financial forecasting [66]. Despite their predictive power, these models are frequently criticized as unexplainable because they do not transparently reveal their internal decision-making processes [69, 99, 11]. In many high-stakes domains—such as hiring, loan approvals, or medical diagnostics—simply achieving high predictive accuracy is not enough as it is important to understand why a particular decision was made.

Global explanation, e.g., overall feature importance [68, 70], capture general trends in a model's behavior, yet they fail to reveal instance-specific factors that drive individual predictions. Local explanations focus on a single instance x_{test} whose prediction we want to understand [12, 36]. One common approach to local explanations is to build a simple, interpretable model, such as a linear or rule-based predictor, that approximates the behavior of the global or target classifier $f(\cdot)$ in a neighborhood around x_{test}. The local model's coefficients or rules then provide feature attributions or explanations. Among the most widely recognized approaches that build local models as explanations are Local Interpretable Model-Agnostic Explanations (LIME) [95] and its variants.

Despite its popularity due to simplicity and model-agnostic nature, LIME-based explanation suffers from **instability** and **poor fidelity**. Its reliance on random perturbations makes the generated explanations sensitive to minor variations (instability), and its focus on the test instance, in terms of biased sampling that favors points near the test instance, can miss the true decision boundary of the target model, leading to inaccurate local approximations (poor fidelity) [121, 8, 48, 112].

Recent advances work on these issues. GLIME [121] enhances stability by integrating weights into the sampling distribution, improving convergence. CALIME [31] enforces causal relationships between features in synthetic sampling, and USLIME [101] uses uncertainty sampling to target low-confidence regions. However, these approaches rely on random or heuristic sampling, which may not balance points across the decision boundary, especially for test instances far from it. To better capture the decision boundary, a local model requires balanced data points on both sides.

To address these limitations, we propose a novel approach, **Adversarially Bracketed Local Explanation (ABLE)** , for constructing local models with high fidelity and stability. The key insight of our approach is to explicitly generate data points that bracket the *local decision boundary* for x_{test}. We emphasize that the local decision boundary for x_{test} is a small segment of the target decision boundary that surrounds the projection of x_{test} on the decision boundary. In other words, the local decision boundary for x_{test} is part of the target decision boundary that is nearest to and thus has the most influence on x_{test}. The data points generated by our approach lie very close to but on opposite sides of the local decision boundary for x_{test}. By training a local model on these points, we explicitly target the local decision boundary, resulting in a more stable and faithful representation. Figure 3.1 shows that without bracketing, a local model could have excessive flexibility, allowing it to match the target model's predictions without truly aligning with the target

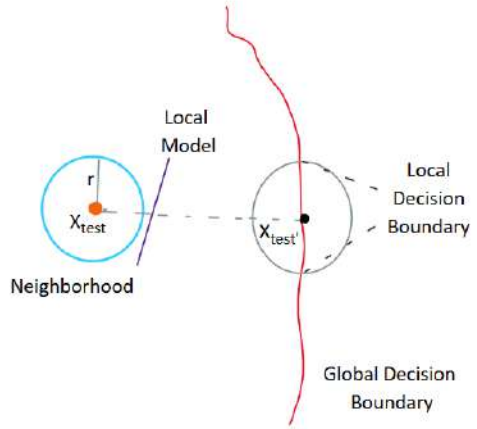

Figure 3.1: In this example, x_{test} is located far away from the global decision boundary. The local model approximates the behavior of the target model in a small neighborhood of x_{test}, in terms that it produces the same classifications for the neighborhood data points as the target model. However, the local model (purple line) does not align with the local decision boundary (red curve) of the target model for x_{test}.

decision boundary.

Specifically, our ABLE approach consists of three major steps. First, we sample the points in a small neighborhood of x_{test} by adding bounded Gaussian noise to x_{test}. Second, for each neighborhood point D, we construct a pair of adversarial points A and A'. An adversarial point is a minimally perturbed version of another data point that causes the model to produce a different prediction. We create the pair using a two-step procedure: (i) we apply a (forward) adversarial attack to D to produce A, which lies on the opposite side of the local decision boundary than D, and (ii) we perform a (reverse) attack on A to obtain A', which lies on the opposite side of the local decision boundary than A but the same side as D. Third, we train a local model using these adversarial pairs to approximate the local decision boundary for x_{test}.

We evaluate the effectiveness of our approach on six benchmark datasets: Credit Default [134], Adult Income [14], Breast Cancer [129], Mushroom [2], Car Evaluation [18], and Covertype [17]—using three distinct neural network architectures (MLP, TabNet [9], and TabTransformer [55]) as target models, resulting in a total of eighteen target models. Moreover, for each of these eighteen models we test all four ABLE variants (ABLE_PGD, ABLE_FGSM, ABLE_etDF, and ABLE_HSJ)), for a comprehensive evaluation of how different adversarial-pair generation approaches impact fidelity and stability. Across these 18 configurations, ABLE achieved the highest fidelity in 16 cases, outperforming all baselines including LIME [95], GLIME [121], USLIME [101], and CALIME [31]. In terms of stability, ABLE approaches ranked best in 12 out of 18 cases, with 4 cases resulting in ties with the top-performing baseline.

Our major research contributions are summarized as follows:

1. **Approach:** We propose a novel approach to constructing local models for explaining model pre-

dictions. Our work is the first to apply adversarial pairs as anchors for local model construction, leading to improved fidelity and stability over random sampling.

2. **Evaluation:** We performed an experimental evaluation of our approach. The results indicate that our approach can construct local models of higher fidelity and stability than the state-of-the-art.

3. **Tool**: We implemented our approach in a publicly available tool, which can be accessed at [4].

We note that our approach can be considered as a general approach for constructing local models of high fidelity and stability.

3.2 Related Work

The need to explain machine learning models has produced two paths: global approaches that reveal dataset-level patterns and local approaches that detail individual predictions. Global approaches, including feature importance scores [36], partial dependence plots [82], and model distillation [70, 52]—capture broad trends but do not explain why a model makes individual decisions. Local explanation approaches, including ours, focus on explaining individual decisions by approximating the model behavior around individual instances [95, 60].

LIME [95] introduced the idea of training simple local models on perturbed data points near a target instance. However, LIME's reliance on random perturbations leads to instability, as many data points miss critical regions of the decision boundary [113, 43]. GLIME [121] improved this approach by weighting perturbations to favor regions near the decision boundary. Still, it relies on stochastic sampling. More importantly, they focus on the test instance, in terms of weighted sampling that favors data points close to the test instance. The decision boundary of the local model they construct does not necessarily align with the target decision boundary. CALIME [31] improves fidelity by generating synthetic neighborhoods that respect causal relationships in tabular data. The causal relationships are inferred from a reference dataset, which may not be readily available in practice. USLIME [101] leverages the target model's prediction confidence to drive sampling: points near the model's uncertainty regions are preferentially chosen, improving boundary coverage. Despite this focus, USLIME still uses heuristic selection criteria and may undersample regions if uncertainty estimates are poor. In contrast, our approach directly focuses on the target decision boundary by generating adversarial pairs that bracket the local decision boundary for the test instance. We then train a local model using those pairs, leading to higher stability and fidelity.

Another line of research improves LIME by utilizing training data to improve the quality of perturbations. For example, AutoLIME [107] trains an autoencoder on the training set and performs perturbations in the latent space, thereby generating synthetic data points that better capture the data distribution. Similarly, DLIME [135] uses the training data to deterministically select local data points via hierarchical clustering and k-nearest neighbors, which reduces the instability inherent in random perturbations. In contrast, our approach relies solely on model queries to generate adversarial pairs that bracket the decision boundary—a key advantage when training data

are unavailable.

SHAP [76] contributes to the local explanation field by utilizing Shapley values from game theory to assign feature attributions. While SHAP's theoretical guarantees are strong, computing exact Shapley values is very expensive since it requires evaluating all possible subsets of features. KernelSHAP approximates Shapley values by sampling feature subsets, but does not explicitly capture the decision boundary of the target model [76].

An alternative perspective is to use inherently interpretable models for explanations [99]. Rather than relying on post-hoc approaches to explain a complex model's behavior, these approaches advocate designing models that are transparent by nature [99, 123]. Models such as rule-based systems [125], decision trees [19], and generalized additive models [49] integrate interpretability into their very structure. However, while these models are inherently interpretable, they struggle to capture more complex relationships in high-dimensional or non-linear data [70].

Research on adversarial perturbations [118, 47, 78] showed that small changes can move a model's input across the decision boundary. Recent studies [43, 28] use adversarial examples to locate boundary regions, although their main goal is vulnerability detection rather than explanation. Adversarial perturbations have also been used in other areas such as fairness testing [137] and robustness training [105]. In our work, we generate adversarial pairs that straddle the local decision boundary and use them as anchor points to train a local model for explaining individual model decisions.

3.3 Background

In this section, we review the basic concepts in local explanation approaches (with a focus on local models), the key characteristics of effective local explanations, and adversarial data points.

3.3.1 LIME-Based Explanations

LIME [95] and its variants [121, 31, 101] approximate the behavior of a target model with a simpler, interpretable surrogate model in a small neighborhood of a specific instance. At a high level, these approaches perturb $\mathbf{x}_{\text{test}}$ to generate nearby data points and fits a weighted linear model, where the resulting coefficients indicate feature importance. The quality of the linear model depends to a significant extent on the quality of the sampled data points. Several variants such as GLIME [121], CALIME [31], USLIME [101] and others [135, 107] have been developed to improve the sampling strategy and thus the quality of the explanations.

A useful local explanation should exhibit two critical properties:

Stability: The explanation should be robust to small perturbations in $\mathbf{x}_{\text{test}}$ [95, 121, 36]. This means that adding minor noise or changing the sampling seed should not result in significantly different explanations.

Fidelity: The local model must accurately capture the behavior of the target model in the immediate vicinity of $\mathbf{x}_{\text{test}}$ [36, 121]. This ensures that the explanation faithfully represents the

model's decision-making process near the instance of interest.

3.3.2 Adversarial Data Points

Given a data point, $\mathbf{x}$, and a model f, adversarial data points are constructed by identifying the smallest perturbation, δ, such that:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \delta, \quad \text{with } f(\mathbf{x}_{\text{adv}}) \neq f(\mathbf{x}).$$

Because δ is minimal, these data points lie close to the target decision boundary, revealing the precise location where $f(\cdot)$ transitions between classes [118, 78, 46]. Adversarial attacks can be untargeted, where the goal is simply to perturb to any class, or targeted, where the goal is to make the model predict a specific different class.

Many adversarial attack methods have been proposed to generate adversarial data points, including Fast Gradient Sign Method (FGSM) [46], Gradient Descent (PGD) [77], DeepFool [83], and ET-DeepFool [65]. Our approach is agnostic to the attack method and can work with any of them, as demonstrated in Section 3.6.

3.4 Motivation

Although LIME is popular for its simplicity and model-agnostic nature, it suffers from two major shortcomings, as highlighted in [121].

Instability LIME generates local perturbations by randomly sampling points around the test instance x_{test}, resulting in varying sample sets and inconsistent explanations. Additionally, LIME assigns weights to each perturbation based on an exponential kernel function defined as $\exp\left(-\frac{\|x-x_{\text{test}}\|_2^2}{\sigma^2}\right)$, where σ is the kernel width controlling the locality of explanations. Tan et al. [121] showed that small σ values (e.g., 0.25), sharply reduce weights for slightly distant points, diminishing their influence and increasing instability.

Poor Fidelity For a local model to be faithful, it must capture the target model's behavior where predictions change— near the decision boundary [28, 43]. Data points near the decision boundary provide crucial insights into how small changes in input lead to label flips. LIME's broad, unguided sampling tends to perturb $\mathbf{x}_{\text{test}}$ randomly in all directions. Furthermore, LIME primarily samples data points from the neighborhood around x_{test} and its proximity weighting scheme favors those data points. If x_{test} is far from the decision boundary, the sampled points in its vicinity are also likely to be far from the decision boundary (see Figure 3.1). This can cause the local model to misrepresent the local decision-making process.

The major challenge in building a local model of high fidelity and stability is how to faithfully replicate the decision boundary of the target classifier. As shown in Figure 3.2.I, when a test instance X_{test} (shown in orange) is perturbed within a neighborhood of radius r, the majority of

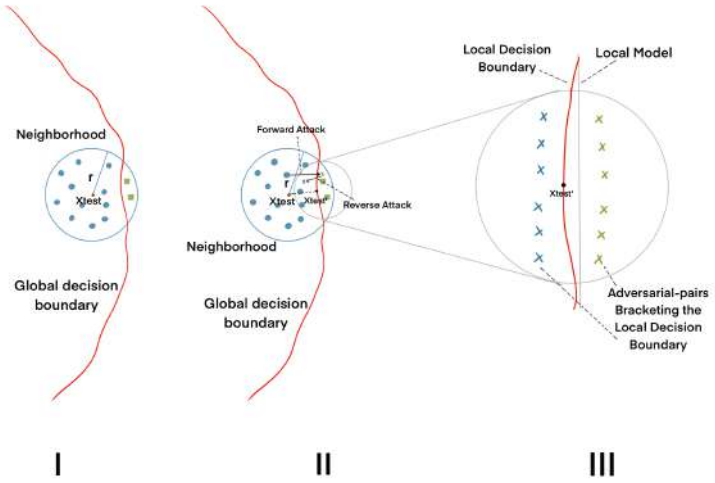

Figure 3.2: (I) X_{test} is surrounded by neighborhood points (blue dots for same class and green boxes for different class) within radius r (red: global decision boundary). (II) $X_{test'}$ is the projection onto the decision boundary; forward and reverse adversarial attacks generate adversarial pairs. (III) Adversarial pairs (blue crosses for the same class and green crosses for a different class) bracket the local decision boundary and are used to train the local model.

the data points (blue dots) may fall on the same side of the decision boundary (red line), while only a few (green dots) lie on the opposite side. If X_{test} is far from the decision boundary, all neighborhood data points may belong to one side (e.g., see Figure 3.1). Such imbalanced sampling leads to incomplete coverage of the decision boundary. Without enough data points from both sides of the boundary, the local model is trained on an incomplete view of the decision boundary region. As a result, it may not faithfully replicate the decision boundary.

To overcome this challenge, we explicitly target the space around the *local decision boundary* that most influences X_{test}. We generate pairs of data points that bracket this boundary. As illustrated in Figure 3.2.II, for each neighborhood instance, a forward PGD attack minimally perturbs the point until its predicted label flips, effectively pushing it to the other side of the decision boundary. A reverse PGD attack then perturbs the flipped point back to its original side. This two-step process creates an adversarial pair that tightly brackets the target boundary.

Figure 3.2.III further shows the result of our approach. Here, adversarial pairs (marked with X symbols) are shown for multiple neighborhood data points. This targeted sampling strategy ensures that the local model is trained on data that covers both sides of the critical decision boundary region, resulting in a robust local approximation of the classifier's behavior.

3.5 Approach

In this section, we present our **Adversarially Bracketed Local Explanation (ABLE)** approach for constructing a local model around a specific test instance $\mathbf{x}_{\text{test}}$. The fundamental idea is to generate a set of adversarial pairs that bracket the target decision boundary rather than relying

on random sampling. We first describe the approach for binary classification, then discuss the multi-class extension in a separate subsection.

3.5.1 Problem Setup and Notation

Let $f : \mathbb{R}^d \to \{0, 1\}$ be a binary classifier that assigns one of two classes to any input in $\mathbb{R}^d$, where d is the feature dimensionality. We aim to explain $f(\mathbf{x}_{\text{test}})$ by training a local model $g(\mathbf{x})$ in the neighborhood of radius r around $\mathbf{x}_{\text{test}}$:

$$\mathcal{N}(\mathbf{x}_{\text{test}}, r) = \left\{ \mathbf{x} \in \mathbb{R}^d \,\middle|\, \|\mathbf{x} - \mathbf{x}_{\text{test}}\|_2 \leq r \right\}.$$

We assume that the target model $f(\cdot)$ allows us to retrieve prediction probabilities for any input $\mathbf{x}$. Our local model is an interpretable logistic regression classifier of the form:

$$g(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function, and $(\mathbf{w}, b)$ are the model parameters. We train $g(\cdot)$ to approximate the decision boundary near $\mathbf{x}_{\text{test}}$, and interpret the learned coefficients to understand local feature importance. The multi-class extension is discussed in Subsection 3.5.6.

3.5.2 Overview of Our Approach

Our approach, ABLE, has three main steps:

1. **(Step 1) Neighborhood Generation:** We sample a set of points $\{\mathbf{x}_i\}_{i=1}^n$ in the vicinity of the test point $\mathbf{x}_{\text{test}}$ by adding bounded random noise to $\mathbf{x}_{\text{test}}$.
2. **(Step 2) Adversarial Pair Generation:** For each $\mathbf{x}_i$, we apply an adversarial attack method to find a minimal perturbation that changes the classifier's prediction, resulting in an adversarial point $\mathbf{x}_i^{\text{adv}}$. We then apply a reverse perturbation process to return $\mathbf{x}_i^{\text{adv}}$ to its original label, forming a pair $(\mathbf{x}_i^{\text{adv}}, \mathbf{x}_i^{\text{rev}})$. These two points lie on opposite sides of the decision boundary.
3. **(Step 3) Local Model Training:** We train a logistic regression model on these adversarial pairs to capture the local decision boundary.

Each neighborhood point $\mathbf{x}_i$ generates an adversarial pair that brackets the local decision boundary, providing boundary-focused coverage. In the following subsections, we detail each step.

3.5.3 Step 1: Neighborhood Generation

We first generate a set of n data points in the neighborhood of the test point $\mathbf{x}_{\text{test}}$. For each sample, we draw a noise vector $\mathbf{z}_i$ from a standard multivariate Gaussian distribution, $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. To ensure that the perturbation remains within a prescribed radius r, we rescale the noise by computing

$$\delta_i = \mathbf{z}_i \cdot \min\left(1, \frac{r}{\|\mathbf{z}_i\|_2}\right),$$

ensuring that $\|\delta_i\|_2 \leq r$. The neighborhood sample is then obtained by adding this perturbation to the original test point, i.e., $\mathbf{x}_i = \mathbf{x}_{\text{test}} + \delta_i$, and the corresponding label is determined by evaluating the classifier f at $\mathbf{x}_i$, so that $y_i = f(\mathbf{x}_i)$. This process results in the neighborhood dataset $\mathcal{D}_{\text{neighbors}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$. We include the original test point $\mathbf{x}_{\text{test}}$ in this neighborhood dataset. Unlike pure random-sampling approaches, our method uses each $\mathbf{x}_i$ as a starting point for generating adversarial pairs. These pairs lie on opposite sides of the target decision boundary, effectively bracketing the boundary and capturing the critical transition region.

3.5.4 Step 2: Adversarial Pair Generation

For each neighborhood point $(\mathbf{x}_i, y_i)$ from Step 1, we generate an adversarial pair that tightly brackets the classifier's decision boundary. First, starting from $\mathbf{x}_i$, we apply an adversarial attack to obtain

$$\mathbf{x}_i^{\text{adv}} = \mathbf{x}_i + \delta_i^{\text{adv}},$$

such that $f(\mathbf{x}_i^{\text{adv}}) \neq y_i$. To find the minimal perturbation δ_i^{adv}, we begin with a conservative default budget. We then execute the attack, whether single-step or iterative. If no adversarial example is found, the budget is increased incrementally (e.g., by 0.1) until success. Attack methods that automatically compute minimal perturbations determine δ_i^{adv} without a preset bound.

Next, from $\mathbf{x}_i^{\text{adv}}$, we apply a second adversarial attack to return to the original class:

$$\mathbf{x}_i^{\text{rev}} = \mathbf{x}_i^{\text{adv}} + \delta_i^{\text{rev}},$$

such that $f(\mathbf{x}_i^{\text{rev}}) = y_i$. When there are only two classes, flipping the prediction from $\mathbf{x}_i^{\text{adv}}$ naturally returns to y_i. This ensures that the resulting pair brackets the decision boundary. Each neighborhood point thus results in one adversarial pair $(\mathbf{x}_i^{\text{adv}}, \mathbf{x}_i^{\text{rev}})$.

3.5.5 Step 3: Local Model Training

After generating adversarial pairs for all n neighborhood points, we obtain

$$\mathcal{D}_{\text{adv}} = \left\{ (\mathbf{x}_i^{\text{adv}}, f(\mathbf{x}_i^{\text{adv}})),\ (\mathbf{x}_i^{\text{rev}}, y_i) \right\}_{i=1}^n,$$

where $\mathbf{x}_i^{\text{adv}}$ is labeled with $f(\mathbf{x}_i^{\text{adv}})$, and $\mathbf{x}_i^{\text{rev}}$ with the original $y_i = f(\mathbf{x}_i)$.

We then train a binary logistic regression model by minimizing the cross-entropy loss on $\mathcal{D}_{\text{adv}}$. Because the adversarial examples lie close to the true decision boundary, this linear surrogate closely approximates the local behavior of f.

3.5.6 Multi-Class Extension

For multi-class classification, where $f : \mathbb{R}^d \to \{1, 2, \ldots, C\}$ with $C > 2$, the approach requires minor adaptations. Steps 1 and 3 are largely unchanged, but Step 2 and the local model formulation are

adjusted as follows.

In Step 2, the initial adversarial attack remains untargeted, yielding $\mathbf{x}_i^{\text{adv}}$ with $f(\mathbf{x}_i^{\text{adv}}) \neq y_i$. However, for the reverse attack, we use a targeted attack to explicitly direct the perturbation back to the original class y_i, as an untargeted flip might land in a third class. This ensures the pair brackets the boundary between y_i and the perturbed class.

In Step 3, we replace binary logistic regression with a multinomial logistic (softmax) model [84, Eq. (8.33)]:

$$g_c(\mathbf{x}) = \frac{\exp\left(\mathbf{w}_c^\top \mathbf{x} + b_c\right)}{\sum_{k=1}^{C} \exp\left(\mathbf{w}_k^\top \mathbf{x} + b_k\right)}, \quad c = 1, \ldots, C,$$

and fit the parameters (W, b) by minimizing the cross-entropy on $\mathcal{D}_{\text{adv}}$.

Table 3.1: The table summarizes the number of instances, categorical and continuous features, and class distributions for the datasets.

Dataset	Instances	Categorical	Continuous	Classes
Credit Default	30,000	23	0	2
Adult Income	48,842	8	6	2
Breast Cancer	569	0	30	2
Mushroom	8,124	22	0	2
Car Evaluation	1,728	6	0	4
Covertype	581,012	10	44	7

3.6 Experiment Design

In this section, we present the experimental setup used to evaluate our proposed approach, *ABLE*, in constructing local models. Our primary objectives are:

1. **RQ1: Fidelity.** How accurately does ABLE replicate the target model's behavior, as measured by the R^2 score between the target and local model outputs?
2. **RQ2: Stability.** How consistent are ABLE's explanations, i.e., important features, under small perturbations of the test instance, as measured by the Jaccard Index on the top-K features?
3. **RQ3: Hyperparameter Impact.** How do variations in the neighborhood radius and number of adversarial pairs affect ABLE's performance?

3.6.1 Datasets

We evaluate ABLE on six benchmark datasets from the UCI Machine Learning Repository, commonly used in tabular classification tasks: Credit Default [134], Adult Income [14], Breast Cancer [129], Mushroom [2], Car Evaluation [18], and Covertype [17]. These datasets vary in size, feature

types, and class distributions, as summarized in Table 3.1. To preprocess the data, we use the RDT HyperTransformer [88] to transform both categorical and continuous features into a unified numerical representation while preserving the original number of features. The HyperTransformer automatically detects feature types and applies appropriate transformers: categorical features are numerically encoded using methods such as label encoding, which maps each category to a unique integer, and continuous features are scaled using transformers like FloatFormatter or GaussianNormalizer. Missing values are handled using strategies such as mean imputation or random sampling, as configured by the HyperTransformer. The transformed features are then standardized using a StandardScaler to achieve zero mean and unit variance, ensuring compatibility with downstream model training and evaluation. Each dataset is split into training (70%), validation (15%), and test (15%) sets, following standard practices in tabular machine learning [64].

Table 3.2: Test accuracy of target models on 15% test data, trained on 70% train data with 15% validation split.

Dataset	**Model Accuracy**		
	MLP	**TabNet**	**TabTransformer**
Adult Income	0.845	0.834	0.847
Breast Cancer	0.982	0.930	0.930
Credit	0.816	0.817	0.817
Mushroom	0.991	0.994	0.988
Car Evaluation	0.846	0.876	0.715
Covertype	0.815	0.884	0.921

3.6.2 Target Models

We train three neural network classifiers for each dataset to serve as target models:

1. **MLP Classifier.** A fully connected neural network with two hidden layers, each followed by ReLU activation.
2. **TabNet Classifier [9].** A sequential attention-based model designed for tabular data, leveraging feature-wise dynamic attention to identify relevant patterns.
3. **TabTransformer Classifier [55].** A transformer-based model with multi-head self-attention for feature encoding in tabular data.

Each model is trained on the training set, tuned on the validation set, and evaluated on the test set. Test accuracy for each model is reported in Table 3.2.

3.6.3 Adversarial Sample Generation

We generate adversarial pairs using four attack variants: PGD [77], FGSM [46], an enhanced DeepFool variant (etDF) [65], and HopSkipJump (HSJ) [27]. PGD, FGSM, and etDF are white-box, gradient-based attacks, while HSJ is a black-box, query-based method that does not require access to model internals. All attacks are implemented using the Adversarial Robustness Toolbox (ART) [86], except etDF, which uses the authors' official implementation [65].

For each test instance, we sample a neighbor $\mathbf{x}_i$ and apply the chosen attack to generate an adversarial point $\mathbf{x}_i^{\text{adv}}$ such that $f(\mathbf{x}_i^{\text{adv}}) \neq y_i$. The procedure varies by each attack method:

1. **ABLE_FGSM:** Applies a single-step gradient update using ART's default ϵ, which defines the radius of the perturbation ball. If no adversarial example is found, ϵ is incremented by 0.1 until success.
2. **ABLE_PGD:** A multi-step variant that builds on FGSM by performing iterative gradient-based updates within the same ϵ-ball. If no adversarial example is generated, ϵ is similarly increased by 0.1.
3. **ABLE_etDF:** Uses an adaptive DeepFool procedure to compute the minimal perturbation needed to reach the decision boundary, without requiring a fixed ϵ.
4. **ABLE_HSJ:** A black-box attack that estimates the decision boundary via query-based sampling and refines the perturbation using binary search.

3.6.4 Hyperparameters

To investigate the impact of hyperparameters on the effectiveness of our approach, we conducted an analysis on the following two key hyperparmeters configurations:

1. **Neighborhood Radius (r):** This parameter defines the local region around x_{test} from which the neighborhood points are sampled. We vary r across the values $\{0.2, 0.4, 0.6, 0.8, 1.0\}$.
2. **Number of Neighborhood Points (n):** We vary n, the number of points sampled within $\mathcal{N}(x_{\text{test}}, r)$. We vary n across the values $\{50, 75, 100, 150\}$

For each configuration, we generate adversarial pairs, train the local model, and evaluate its fidelity.

3.6.5 Baselines

We compare *ABLE* against four local explanation baselines:

1. **LIME** [94]. Uses random perturbations and a weighted linear regressor to approximate the local decision boundary of a target model.
2. **GLIME** [121]. Improves LIME stability by integrating the weighting function into the sampling distribution, reducing the reliance on post-hoc weighting.

3. **USLIME** [101]. Extends LIME by using uncertainty sampling to select neighborhood points close to the decision boundary, based on a boundary score from prediction probabilities and distance filtering, improving fidelity on tabular data.
4. **CALIME** [31]. Enhances LIME by generating causal-aware neighborhoods using NCDA and GENCDA, improving fidelity and stability. It requires a reference dataset for causal discovery. In this experiment, we use the training dataset as the reference dataset. We note that a reference dataset may not be available in practice. Other methods do not require a reference dataset.

Table 3.3: Effect of Neighborhood Radius (r) and Number of Neighborhood Points (n) on Local Fidelity. Higher values indicate better fidelity.

Neighborhood Radius (r)	**Number of Neighborhood Points** (n)			
	50	**75**	**100**	**150**
0.2	0.840	0.870	0.889	0.908
0.4	0.834	0.862	0.881	0.904
0.6	0.834	0.863	0.880	0.902
0.8	0.829	0.857	0.875	0.896
1.0	0.827	0.854	0.870	0.890

Table 3.4: Fidelity Comparison. Higher $\mathbf{R^2}$ indicates better alignment with the target model.

Dataset	Classifier	ABLE_PGD	ABLE_etDF	ABLE_FGSM	ABLE_HSJ	LIME	GLIME	USLIME	CALIME
Credit	MLP	**0.983**	0.964	0.900	0.948	0.412	0.407	0.361	0.313
	TabNet	0.759	**0.866**	0.837	0.723	0.831	0.845	0.730	0.661
	TabTransformer	0.954	**0.984**	0.967	0.933	0.385	0.377	0.467	0.164
Breast_Cancer	MLP	**0.959**	0.954	0.932	0.917	0.236	0.236	0.145	0.911
	TabNet	**0.890**	0.876	0.816	0.731	0.388	0.380	0.337	0.888
	TabTransformer	**0.979**	0.977	0.939	0.895	0.501	0.501	0.356	0.879
Mushroom	MLP	0.975	**0.991**	0.971	0.942	0.373	0.350	0.465	0.893
	TabNet	0.888	**0.932**	0.892	0.697	0.588	0.594	0.495	0.872
	TabTransformer	**0.975**	0.959	0.965	0.922	0.394	0.393	0.215	0.753
Covertype	MLP	**0.972**	0.911	0.961	0.863	0.026	0.023	0.133	0.559
	TabNet	**0.863**	0.749	0.788	0.702	0.505	0.510	0.422	0.531
	TabTransformer	**0.898**	0.881	0.868	0.701	0.233	0.229	0.176	0.460
Adult	MLP	0.932	**0.952**	0.928	0.842	0.738	0.757	0.858	0.562
	TabNet	0.752	0.874	0.722	0.744	**0.946**	0.945	0.884	0.456
	TabTransformer	**0.907**	0.872	0.892	0.733	0.554	0.572	0.670	0.358
Car	MLP	**0.986**	0.966	0.961	0.959	0.721	0.665	0.896	0.719
	TabNet	0.817	0.728	0.845	0.853	0.708	0.725	**0.907**	0.551
	TabTransformer	**0.826**	0.728	0.657	0.926	0.489	0.530	0.779	0.489
Average		**0.906**	0.898	0.880	0.831	0.502	0.502	0.516	0.612

3.6.6 Evaluation Metrics

1) Fidelity (R^2 Score). We measure how well the predicted probabilities of the local model match the probabilities of the target model in a local neighborhood of *test* instances. For each x_{test}, we generate a set of neighborhood points by adding bounded Gaussian noise, then split these points into training and evaluation sets. The local model is trained on adversarial pairs constructed from the training set, and its fidelity is evaluated on the separate evaluation set. For baseline approaches, the evaluation set is drawn similarly from their respective local neighborhoods using the prescribed sampling procedure for each method. We compute the coefficient of determination (R^2) between the output of the local model and the target model in this evaluation set:

$$R^2(f, g) = 1 \; - \; \frac{\sum_{x \in x_{\text{test}}} (f(x) - g(x))^2}{\sum_{x \in x_{\text{test}}} (f(x) - \bar{f})^2},$$

where $f(x)$ is the target model's probability estimate, and $g(x)$ is the local model's estimate. A higher R^2 indicates better fidelity [16], which means that the local model more accurately approximates the predictions of the target model [16] in the neighborhood.

2) Stability (Jaccard Index). We assess the stability of each explanation by introducing a small random noise to the test instance to create a single perturbed version. For both the original and perturbed instances, we generate explanations and extract their top-K important features (with $K = 5$ being commonly used in the literature). We then compute the Jaccard Index between these two sets of top-K features:

$$\text{JI}(\mathcal{F}_{\text{original}}, \mathcal{F}_{\text{perturbed}}) = \frac{|\mathcal{F}_{\text{original}} \cap \mathcal{F}_{\text{perturbed}}|}{|\mathcal{F}_{\text{original}} \cup \mathcal{F}_{\text{perturbed}}|},$$

where $\mathcal{F}_{\text{original}}$ and $\mathcal{F}_{\text{perturbed}}$ denote the sets of top-K features from the original and perturbed instances, respectively. A higher Jaccard Index indicates that the explainer consistently identifies the same features as important across the original and slightly perturbed instances [120].

3) Time. We report the total run time in seconds for each approach to generate a local model, from sample/perturbation generation to final model training. Lower time indicates higher efficiency.

3.6.7 Experimental Procedure

We evaluate our approach by first training three target models (MLP, TabNet, and TabTransformer) on the training set, validating on the validation set, and fixing the final models for testing. We select 100 random test instances per dataset and evaluate ABLE with each adversarial attack method alongside the baseline methods. We then assess the fidelity and stability. In addition, we record the total computation time for each method. This entire process is repeated over 10 random seeds, and the resulting R^2, Jaccard Index, and runtime metrics are averaged.

All experiments were performed on a machine with a multi-core processor operating at 2.10 GHz and 32 GB of RAM.

Table 3.5: Stability Comparison. Higher Jaccard Index indicates more consistent feature importance under perturbations.

Dataset	Classifier	ABLE_PGD	ABLE_etDF	ABLE_FGSM	ABLE_HSJ	LIME	GLIME	USLIME	CALIME
Credit	MLP	**0.984**	0.833	0.974	0.883	0.800	0.787	0.795	0.452
	TabNet	**1.000**	0.930	0.882	1.000	0.899	**1.000**	**1.000**	0.429
	TabTransformer	0.956	0.913	0.981	0.983	**1.000**	**1.000**	0.982	0.313
Breast_Cancer	MLP	0.983	**1.000**	0.984	0.867	0.902	0.917	0.562	0.357
	TabNet	0.833	0.889	0.867	0.818	0.938	**0.963**	0.834	0.215
	TabTransformer	**0.989**	0.977	0.957	0.967	0.943	0.963	0.943	0.284
Mushroom	MLP	0.913	**0.970**	0.939	0.917	0.905	**0.970**	0.905	0.424
	TabNet	0.854	**0.950**	0.889	0.767	0.802	0.921	0.905	0.424
	TabTransformer	0.926	0.931	0.931	0.850	**1.000**	0.978	0.857	0.343
Covertype	MLP	0.778	0.833	**1.000**	0.952	0.311	0.273	0.515	0.299
	TabNet	0.833	0.810	0.833	0.848	0.882	0.961	**0.980**	0.376
	TabTransformer	0.714	0.833	**1.000**	0.879	0.730	0.792	0.730	0.349
Adult	MLP	0.956	0.963	0.960	0.967	0.955	**0.988**	0.967	0.478
	TabNet	0.952	0.965	0.939	0.848	**1.000**	**1.000**	**1.000**	0.621
	TabTransformer	0.943	0.944	**0.974**	0.846	0.882	0.793	0.937	0.498
Car	MLP	0.958	**0.985**	0.972	0.967	0.945	0.947	0.960	0.767
	TabNet	0.944	**1.000**	0.889	0.981	0.910	**1.000**	**1.000**	0.899
	TabTransformer	0.958	0.987	**1.000**	0.883	**1.000**	**1.000**	0.972	0.796
Average		0.915	0.928	**0.932**	0.901	0.878	0.903	0.880	0.435

Table 3.6: Runtime Comparison in Seconds (Aggregated Across All Datasets). Lower values indicate better computational efficiency.

Classifier	ABLE_etDF	ABLE_PGD	ABLE_FGSM	ABLE_HSJ	LIME	GLIME	USLIME	CALIME
MLP	0.416	1.367	0.121	2.124	0.111	0.153	0.163	52.345
TabNet	1.432	3.781	0.480	4.140	0.210	0.249	0.420	52.509
TabTransformer	0.602	2.831	0.163	2.441	0.120	0.157	0.147	64.785

3.7 Results and Discussion

3.7.1 Hyperparameter Analysis

We evaluated the sensitivity of the ABLE framework to its primary hyperparameters: the neighborhood radius (r) and the number of neighborhood points (n). Table 3.3 presents the average local fidelity (R^2) across all four ABLE approaches, six datasets, and three classifiers.

Neighborhood Radius (r): Fidelity is highest at a smaller radius (e.g., $r = 0.2$), achieving an R^2 of up to 0.908. This indicates that a tighter, more localized neighborhood allows the local model to better approximate the target model's behavior. As r increases, fidelity declines, suggesting that a larger radius introduces points from regions where the linear approximation is less accurate.

Number of Neighborhood Points (n): Fidelity consistently improves as n increases. However, the gains diminish for larger n, while the computational cost rises. Notably, ABLE achieves

high fidelity (0.908) with just 150 points ($n = 150$).

Based on these findings, we selected $r = 0.2$ and $n = 150$ for all subsequent experiments, balancing high performance with computational efficiency.

3.7.2 Fidelity Comparison

Table 3.4 presents the fidelity results (R^2 scores) comparing ABLE's four variants against the baselines. ABLE consistently and significantly outperforms all baselines in terms of fidelity. On average, ABLE_PGD achieves the highest fidelity with an R^2 score of 0.906, followed closely by ABLE_etDF (0.898), ABLE_FGSM (0.880), and ABLE_HSJ (0.831). In contrast, the baselines perform poorly, with average scores of 0.612 for CALIME and around 0.5 for LIME, GLIME, and USLIME.

This superiority is evident across nearly all configurations. For example, on the Covertype dataset with an MLP target, ABLE_PGD achieves a near-perfect fidelity of 0.972, whereas LIME and GLIME score a mere 0.026 and 0.023, respectively. This demonstrates that by generating data points that bracket the decision boundary, ABLE creates a local model that more accurately reflects the target model's decision-making process. The only exceptions are two instances (TabNet on Adult and Car) where baselines achieve higher scores, though ABLE's overall performance remains strong.

3.7.3 Stability Comparison

Table 3.5 shows the stability results measured by the Jaccard Index. All ABLE variants demonstrate superior stability on average compared to the baselines. ABLE_FGSM leads with an average Jaccard Index of 0.932, followed by ABLE_etDF (0.928), ABLE_PGD (0.915), and ABLE_HSJ (0.901). These scores are higher than those of GLIME (0.903), LIME (0.878), USLIME (0.880), and especially CALIME (0.435).

For instance, on the Covertype dataset with an MLP model, ABLE_FGSM achieves perfect stability (1.000), while LIME and GLIME are highly unstable (0.311 and 0.273). This indicates that by anchoring the local model to the decision boundary, ABLE produces explanations that are more robust to minor input perturbations.

3.7.4 Runtime Analysis

Table 3.6 compares the computational time required to generate a single explanation. As expected, ABLE's iterative adversarial search incurs a higher computational cost than fast sampling methods like LIME, GLIME, and USLIME. ABLE_FGSM is the fastest variant—comparable to LIME, GLIME, and USLIME—due to its single-step attack, while the iterative nature of PGD makes ABLE_PGD the slowest. ABLE_HSJ, being a black-box method, lies in between but is slower than PGD due to repeated boundary estimations. However, ABLE remains significantly faster than CALIME, which incurs substantial overhead from its causal discovery phase. Despite the relative

increase in cost, all ABLE variants complete in just a few seconds per explanation, making them practical for real-world use cases.

The increased runtime for ABLE is a direct trade-off for its substantially higher fidelity and stability. Notably, ABLE_FGSM is highly query-efficient, requiring only 300 queries to target model (150 neighborhood points, yielding 300 training points for the local model) compared to the 5000 points typically used by baselines like LIME and GLIME. This efficiency, combined with superior performance, makes ABLE—particularly ABLE_FGSM—a favorable choice in scenarios where querying the target model is expensive or high fidelity is critical.

3.7.5 Discussion

The performance differences among ABLE's variants highlight the role of the underlying adversarial attack. Iterative white-box approaches like PGD and etDeepFool generally achieve the highest fidelity, likely by finding points closer to the true boundary using gradients. The single-step FGSM, while faster, sometimes results in slightly lower fidelity. The black-box HSJ variant offers comparable stability and fidelity without requiring model internals, making it suitable for scenarios with limited access to the classifier. This suggests that even a query-based approximation of the boundary is more effective than unguided random sampling. The results also suggest that for some complex models like TabNet, finding minimal perturbations can be challenging, which may affect performance. This points to future work in developing even more robust adversarial search techniques tailored for local explanation.

3.8 Conclusion and Future Works

In this paper, we introduced our approach, ABLE, that constructs a local model to explain a model prediction by directly targeting the local decision boundary. By generating pairs of adversarial data points—one that minimally perturbs the test instance to flip the model's prediction and another that reverts it back—we effectively bracket the local decision boundary of the target model for the test instance. Our experimental results demonstrate that ABLE not only mitigates the instability issues inherent in techniques like LIME but also enhances the fidelity of local models by anchoring them around the local decision boundary for the test instance.

Future work includes evaluating ABLE on broader model types and data domains, such as images and text. When applied to image data, our approach must take into account the perceptual properties and spatial structure of the images. Applying perceptually motivated distance metrics like LPIPS [139] or SSIM [128] during adversarial perturbations can help preserve spatial coherence.

Acknowledgment

This work is supported by a research grant (70NANB21H092) from Information Technology Lab of National Institute of Standards and Technology (NIST).

Disclaimer: Certain equipment, instruments, software, or materials are identified in this paper in order to specify the experimental procedure adequately. Such identification is not intended to imply recommendation or endorsement of any product or service by NIST, nor is it intended to imply that the materials or equipment identified are necessarily the best available for the purpose.

Chapter 4

DD-CAM: Minimal Sufficient Explanations for Vision Models Using Delta Debugging

This chapter is based on: K. Khadka, Y. Lei, R. N. Kacker, D. R. Kuhn, "DD-CAM: Minimal Sufficient Explanations for Vision Models Using Delta Debugging," targeting NeurIPS 2026.

Abstract

We introduce a gradient-free framework for identifying minimal, sufficient, and decision-preserving explanations in vision models by isolating the smallest subset of representational units whose joint activation preserves predictions. Unlike existing approaches that aggregate all units, often leading to cluttered saliency maps, our approach, DD-CAM, identifies a 1-minimal subset whose joint activation suffices to preserve the prediction (i.e., removing any unit from the subset alters the prediction). To efficiently isolate minimal sufficient subsets, we adapt delta debugging, a systematic reduction strategy from software debugging, and configure its search strategy based on unit interactions in the classifier head: testing individual units for models with non-interacting units and testing unit combinations for models in which unit interactions exist. We then generate minimal, prediction-preserving saliency maps that highlight only the most essential features. Our experimental evaluation demonstrates that our approach can produce more faithful explanations and achieve higher localization accuracy than the state-of-the-art CAM-based approaches.

4.1 Introduction

Deep Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) have achieved state-of-the-art performance in vision tasks such as image classification, object detection, and segmentation [62, 37]. Despite this success, these models remain largely uninterpretable, raising concerns in

high-stakes domains like healthcare, finance, and autonomous systems [11, 54].

Class Activation Mapping (CAM) is a widely used post-hoc technique that generates saliency maps by leveraging internal representations from vision models. CAM-based approaches use feature maps (in CNNs) or patch tokens (in ViTs), typically from the final layer before classification—because these representations balance high-level semantic content with spatial structure suitable for localization. Gradient-based CAM approaches like Grad-CAM [104] weight features using gradients, while gradient-free alternatives like Score-CAM [126] and Ablation-CAM [35] use forward-pass perturbations. However, all of the existing approaches aggregate contributions from every unit, often producing cluttered saliency maps that obscure which features are truly necessary for the prediction.

We address the above limitation by reframing explanation generation as identifying *minimal sufficient subsets*: a smallest set of representational units whose joint activation preserves the model's prediction. A representational unit (feature map or patch) is necessary with respect to the set if removing it from the set changes the prediction; here, necessity is defined in the local sense, meaning the unit is required for this particular set to preserve the prediction. A set is sufficient if activating only that set preserves the prediction. This minimal sufficiency-based formulation ensures that each selected unit is locally necessary and produces focused explanations that highlight only the essential regions driving the prediction. Furthermore, it generalizes across architectures: whether a model uses convolutions or self-attention, we can uniformly ask which minimal subset of representational units preserves the prediction.

We adapt delta debugging, a systematic reduction strategy from software debugging originally designed to isolate minimal failure-inducing inputs [136], to identify minimal sufficient subsets in vision models. In software debugging, the algorithm begins with a known failure and attempts to find a set of inputs that still preserves that failure. In our setting, the model's prediction plays the same role as the outcome to preserve; instead of preserving a program failure, we aim to preserve the model's prediction. Representational units (feature maps or patch tokens) serve as testable components. The algorithm recursively partitions candidate units and tests whether subsets preserve the prediction, ultimately identifying a 1-minimal set where removing any single unit changes the prediction.

We optimize the performance of the delta debugging algorithm based on the classifier head employed in a vision model. For models where units interact in the classifier head — such as CNNs with multiple fully connected (FC) layers and ReLU activations (e.g., VGG models [110]) or ViTs where self-attention creates dependencies between patches, the general delta debugging algorithm is used. For models where units do not interact in the classifier head, i.e., they contribute to the model prediction independently, such as CNNs with Global Average Pooling (GAP) followed by a single fully connected layer (e.g., ResNet-50 [50], EfficientNet-B0 [119], Inception-v3 [117]), we configure the delta debugging algorithm to test each unit individually, i.e., without considering their combinations, reducing the computation complexity. Both configurations provide the same 1-minimality guarantee.

We evaluated our approach by examining both explanation faithfulness and localization accuracy. For faithfulness, we used eight ImageNet-pretrained models [100], including six CNNs and two vision transformers, and generated explanations for 2,000 ImageNet validation images [100]. We compared DD-CAM against seven state-of-the-art CAM-based methods. Our results show that DD-CAM outperforms all baselines in 15 out of 18 evaluations, averaged across model groups, providing more faithful explanations across both convolutional and transformer models.

For localization accuracy, we conduct a separate evaluation on 1,000 radiologist-annotated chest X-rays from NIH ChestX-ray14 [127] and compared to the same baselines. DD-CAM achieves significantly higher localization accuracy in terms of improving IoU by 45% and precision by 22% over the strongest baseline while producing compact single-region explanations.

Our contributions are threefold. (1) We introduce a gradient-free approach for identifying minimal decision-preserving representational units, providing the first application of delta debugging to vision model explanations. (2) We perform extensive evaluations to show that minimal sufficient explanations improve both faithfulness and localization. (3) We release an implementation, *DD-CAM*, which is anonymously available for review [1].

4.2 Related Work

We review post-hoc, attribution-based explanation methods, beginning with model-agnostic approaches, followed by two model-specific families relevant to our work: sensitivity-based explanations and CAM techniques.

4.2.1 Model-Agnostic Explanations

Model-agnostic methods such as LIME [95], SHAP [76], and related variants [97, 74] explain predictions by fitting surrogate models to perturbed inputs or by estimating Shapley values. Because they operate purely in the input space, they treat the network as a black box and rely on heuristic baselines (e.g., blur, zeros, noise) and masking schemes (e.g., superpixels), which can substantially affect explanation quality and stability [6]. In contrast, we work with internal activations and use zero-masking of representational units as a natural, architecture-consistent baseline.

4.2.2 Sensitivity-Based Explanations

Sensitivity-based methods—including Saliency Maps [109], Guided Backpropagation [115], Integrated Gradients [116], and SmoothGrad [113]—attribute importance using input-output gradients. Although efficient, they often produce noisy attributions due to gradient locality and saturation in deep networks [6], and they do not indicate which internal representations drive the prediction. In contrast, our gradient-free approach identifies minimal subsets of internal units that preserve the output, offering a more directly causal explanation.

4.2.3 Class Activation Mapping Explanations

CAM-based methods leverage internal activation patterns in the final layers of vision models. Grad-CAM [104] and Grad-CAM++ [25] use class-specific gradients to weight representational units, but are affected by gradient saturation and noise. Gradient-free variants such as Score-CAM [126] and Ablation-CAM [35] estimate unit contributions through multiple forward passes, improving robustness but still aggregating all units into dense saliency maps. In contrast, we identify a minimal decision-preserving subset that is sufficient to retain the prediction, yielding stricter explanations focused on essential features. For Vision Transformers, attention-based explainers [26, 5] derive relevance from attention weights, whereas our method again searches for minimal sufficient subsets of patch tokens.

Minimal-input-region methods such as Sufficient Input Subsets (SIS) [23] and Meaningful Perturbation [40] also pursue minimality but operate in the input space and do not produce attribution scores. SIS iteratively removes input features until the confidence drops, while Meaningful Perturbation learns a sparse, smooth mask. In contrast, we enforce minimality over internal representations and still generate attribution maps from the selected units.

4.3 Background

This section introduces internal representations in vision models and delta debugging from software engineering.

4.3.1 Vision Model Representations

Vision models for image classification share a common architecture: a feature extractor producing internal representations, followed by a classifier head mapping these to class predictions.

CNNs process images through convolutional layers, learning hierarchical visual features [62]. The final convolutional layer produces K feature maps $A_k \in \mathbb{R}^{H \times W}$ that retain spatial structure while encoding high-level semantic patterns.

Vision Transformers divide images into patches, embed them via linear projection, and process them through transformer blocks with self-attention [37]. Following ViT-ReciproCAM [22], we extract patch tokens from the first LayerNorm layer of the final transformer encoder block, which produces N patch tokens $P_n \in \mathbb{R}^D$ (typically 196 image patches plus one CLS token). This layer provides unmodified skip-connected features suitable for explainability.

Classifier heads map these representations to predictions. Linear heads use Global Average Pooling followed by a single fully connected layer, where each unit contributes independently. Non-linear heads use multiple layers with ReLU activations, creating interactions between units.

Across architectures, both CNNs and ViTs produce a set of representational units before classification: feature maps or patch tokens. We can uniformly ask: which subset of units is necessary for the prediction?

4.3.2 Delta Debugging

Delta debugging systematically isolates minimal failure-inducing input components in software debugging [136]. Given a program input causing failure, it identifies the smallest subset of components whose removal prevents the failure. The algorithm recursively partitions the input into n subsets and tests each subset and its complement. If a subset or its complement reproduces the failure, the search continues within the subset or its complement. Otherwise, i.e., if none of the subsets or their complements reproduce the failure, the algorithm increases the partition granularity ($n \leftarrow 2n$) and repeats the same process. The algorithm terminates when a 1-minimal set is found, i.e., removing any single component prevents the failure.

4.4 Motivation

Our goal is to identify minimal sufficient explanations: the smallest subset of representational units (feature maps or patch tokens) whose joint activation preserves the model's prediction. A subset $S \subseteq \{1, \ldots, M\}$ is *sufficient* if masking all units outside S preserves the predicted class $\hat{c}$. It is *1-minimal* if removing any single unit from S changes the prediction. While 1-minimality does *not* guarantee the smallest possible set (minimum cardinality), it provides a tractable approximation, as finding the absolute minimal set is NP hard [136]. If multiple 1-minimal subsets exist, we identify one of them and use it to generate explanations [136].

We draw inspiration from fault localization in software debugging, where the goal is to isolate minimal failure-inducing inputs. In our setting, the prediction is the failure to preserve, representational units are testable components, and masking units (setting activations to zero) tests their removal. This analogy motivates adapting delta debugging to vision model explanation.

The concept of minimal sufficient explanation is agnostic to internal representations. Whether a model uses feature maps (CNNs) or patch tokens (ViTs), we can uniformly test subsets for prediction preservation. Only the tested units differ while the algorithmic logic remains unchanged. This enables a unified framework across architectures.

Minimal sufficient explanations offer several advantages: (1) reduced cognitive load by focusing on necessary features [81, 33], (2) causal grounding since each unit is provably necessary, (3) targeted robustness checks in safety-critical applications [36], and (4) visual clarity through less cluttered saliency maps.

4.5 Approach

Let $f : \mathbb{R}^{L \times H \times W} \to \mathbb{R}^C$ be a pretrained vision model, where L is the number of input channels and $H \times W$ is the spatial resolution. An input image is denoted by $I \in \mathbb{R}^{L \times H \times W}$. The model produces class scores $f(I) \in \mathbb{R}^C$, where C is the number of classes and $y_c = f(I)_c$ is the logit for class c. The predicted label is $\hat{c} = \arg\max_c y_c$.

Our objective is to explain the model's decision $\hat{c}$ by identifying a 1-minimal subset of representational units that preserves this prediction. We target the final layer before classification because it balances high-level semantic abstraction with spatial detail, making it suitable for visual interpretability [140]. For CNNs, the final convolutional layer produces $A = [A_1, A_2, \ldots, A_K] \in \mathbb{R}^{K \times h \times w}$, where K is the number of feature maps. For ViTs, we test subsets of the N patch tokens at the input to the final transformer block, while the CLS token is always preserved. The patches are represented as $P = [P_1, P_2, \ldots, P_N] \in \mathbb{R}^{N \times D}$, where D is the embedding dimension. Our goal is to find $S^\star \subseteq \{1, \ldots, M\}$ (where $M = K$ for CNNs or $M = N$ for ViTs) such that activating only the units indexed by $S^\star$ and zero-masking the rest preserves $\hat{c}$. Here, prediction preservation means that the top-1 predicted class remains unchanged. As discussed in Section 4.5.2, unlike input-space masking, zero-masking at the representation level is well-defined.

We introduce a gradient-free approach based on delta debugging to identify $S^\star$. The approach is a three-stage pipeline: (1) Activation Extraction, (2) Subset Selection via Delta Debugging, and (3) Saliency Map Generation.

4.5.1 Step 1: Activation Extraction

Initially, a forward pass is performed with the input image I to compute the original prediction $\hat{c}$ and the representational tensor from the target layer. For CNNs, we extract feature maps $A = [A_1, \ldots, A_K] \in \mathbb{R}^{K \times h \times w}$ from the final convolutional layer using a forward hook. For Vision Transformers, we intervene on the N patch tokens at the input to the final transformer block, following ViT-ReciproCAM [22]. The CLS token is always preserved during masking operations to maintain skip-connection information and ensure proper attention mechanism function. We represent the testable patch tokens as $P = [P_1, \ldots, P_N] \in \mathbb{R}^{N \times D}$.

We split the model at the target layer to reuse cached representations, enabling partial forward passes from the intervention point through the remaining layers. The extracted tensor and prediction $\hat{c}$ are cached for subsequent perturbation tests. We refer to this downstream subnetwork as the remainder network f_{rem}.

4.5.2 Step 2: Subset Selection via Delta Debugging

This stage identifies a minimal subset of representational units $S^\star$ using delta debugging. We employ perturbation-based testing [35], selectively masking (zeroing out) units and observing if the model's prediction $\hat{c}$ remains unchanged.

Unlike masking in the input space, where defining a neutral reference value is challenging, masking at the representation level uses zero as a natural reference value [35], effectively removing the unit's activation. For CNNs, zeroing a feature map A_k sets all activations to zero. For ViTs, masking a patch token P_n sets all token dimensions to zero while preserving the CLS token, ensuring the final self-attention layer operates only on the selected patches. In both cases, zero represents the absence of activation, avoiding artificial patterns and providing a well-defined reference.

Delta Debugging Algorithm

Algorithm 2 adapts the standard delta debugging (DD) procedure to identify a 1-minimal sufficient set of representational units. We begin with the full set $S = \{1, \ldots, M\}$ and an initial granularity $n = 2$. In each recursive call, S is partitioned into n disjoint subsets $S_1, \ldots, S_n$. For each subset S_i, the complement $S \setminus S_i$ is evaluated by zeroing the units in S_i and performing a forward pass through f_{rem}. If the complement preserves the original prediction, then S_i is unnecessary. In this case, delta debugging resets the granularity to $n = 2$ and recurses on the reduced set $S \setminus S_i$. If no subset can be removed at the current granularity, the granularity is increased to $\min(2n, |S|)$ and DD recurses on this larger value of n. When no further reduction is possible (that is, when $n = |S|$ and each single-element complement fails to preserve the prediction), the algorithm returns a 1-minimal sufficient set $S^\star$.

If units interact in the classifier head, masking one unit may influence the contribution of others. In this case, DD must explore complements at increasing levels of granularity through its recursive partition-and-reduction, as shown in Algorithm 2. The standard DD algorithm has $O(M \log M)$ complexity in favorable cases and up to $O(M^2)$ in the worst case when fine partitions are required. When the minimal sufficient set is small, DD often eliminates large groups of units early.

If units do not interact in the classifier head, as in GAP+FC models where the logit for class c decomposes as $y_c(S) = \sum_{k \in S} w_{c,k}\, a_k + b_c$, the decision to discard a unit can be made by a single direct test: mask the unit and check whether the prediction changes, without evaluating any combinations.

However, the one-pass procedure is only correct if we first remove units that work against the predicted class. In a GAP+FC head, each unit k independently adds $(w_{\hat{c},k} - w_{c,k})\, a_k$ to the margin between the predicted class $\hat{c}$ and each alternative class c. If this quantity is non-positive for every alternative class, the unit never helps $\hat{c}$; it either hurts the prediction or has no effect. Removing such a unit from any sufficient set cannot break sufficiency, since the margins can only stay the same or improve (Proposition 1, Appendix 4.10). We discard all such units before the one-pass begins. The remaining units each strictly help $\hat{c}$ against at least one alternative class, so their removal tests are order-independent, and the one-pass correctly yields a 1-minimal sufficient set at $O(M)$ cost.

Illustrative Example. Suppose $S = \{1, \ldots, 8\}$, where units are interacting, and only units 2 and 4 are required to preserve the prediction. DD begins with $n = 2$ and partitions S into $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8\}$. Masking the first subset removes required units and changes the prediction, but masking the second preserves it, so $\{5, 6, 7, 8\}$ is discarded and DD restarts with $S = \{1, 2, 3, 4\}$ and $n = 2$. Masking either half ($\{1, 2\}$ or $\{3, 4\}$) now removes a required unit, so DD increases the granularity to $n = 4$ and tests singletons. Masking unit 1 preserves the prediction, so it is removed and DD restarts with $S = \{2, 3, 4\}$. DD then partitions $\{2, 3, 4\}$ into $\{2, 3\}$ and $\{4\}$, but masking either subset changes the prediction. Increasing to $n = 3$ allows singleton testing again; masking unit 3 preserves the prediction, so it is removed and DD restarts with $S = \{2, 4\}$. With $S = \{2, 4\}$, masking either unit changes the prediction. Since no subset can be removed and the

Algorithm 2 Delta Debugging for Minimal Sufficient Sets

Input: Units $U = \{u_1, \ldots, u_M\}$, remainder network f_{rem}
Output: 1-minimal sufficient set $S^\star$

```
ĉ ← arg max_c f_rem(U)
return DD(U, 2)
function DD(S, n)
    Partition S into n subsets S_1, ..., S_n (sizes differ by at most one)
    removed ← False
    for each S_i do
        S_test ← S \ S_i
        Create U' by zeroing all units not in S_test
        if arg max_c f_rem(U') = ĉ then
            removed ← True
            return DD(S_test, 2)
        end if
    end for
    n_next ← min(2n, |S|)
    if not removed and n_next = n then
        return S
    end if
    return DD(S, n_next)
end function
```

granularity cannot increase further (i.e., $n = |S|$), the algorithm reaches a fixed point and returns the 1-minimal sufficient set $S^\star = \{2, 4\}$.

4.5.3 Step 3: Saliency Map Generation

Once a minimal subset of unit indices $S^\star$ is identified by delta debugging, we compute importance weights for each unit in $S^\star$ based on the drop in the logit for the predicted class $\hat{c}$ when that unit is removed. Let $y_{\hat{c}}$ be the original logit for class $\hat{c}$. For each unit index $i \in S^\star$, we mask the corresponding unit (feature map A_i for CNNs or patch token P_i for ViTs) while keeping the other units in $S^\star$ active, perform a partial forward pass to obtain the modified logit $y'_{\hat{c},i}$, and compute the drop $\delta_i = y_{\hat{c}} - y'_{\hat{c},i}$. The weight w_i is then set to δ_i normalized such that $\sum_{i \in S^\star} w_i = 1$:

$$w_i = \frac{\delta_i}{\sum_{j \in S^\star} \delta_j}$$

This weighting strategy is similar to Ablation CAM [35], as both derive weights from the effect of zeroing out individual units, but differs in normalization. We normalize across the selected minimal subset rather than by the original logit. This ensures that weights reflect the relative importance of units within the identified minimal set.

An interpretable saliency map F_{exp} is generated by computing the weighted average of the

selected units and upsampling to the original input image resolution. For CNNs:

$$F_{exp} = \text{Upsample}\left(\text{Normalize}\left(\sum_{k \in S^\star} w_k A_k\right)\right)$$

For ViTs, the weighted patch tokens are reshaped to the spatial grid and upsampled:

$$F_{exp} = \text{Upsample}\left(\text{Normalize}\left(\text{Reshape}\left(\sum_{n \in S^\star} w_n P_n\right)\right)\right)$$

Here, normalization is to min-max scaling, which maps the values to $[0, 1]$. Bilinear interpolation is commonly used for upsampling. The resulting saliency map F_{exp} highlights the spatial regions most critical for the model's prediction $\hat{c}$, with intensities proportional to unit importance.

Table 4.1: RQ1 Results: Average performance across CNNs with linear classifier heads (ResNet-50, EfficientNet-B0, Inception-v3), CNNs with non-linear classifier heads (VGG-11, VGG-16, VGG-19), and Vision Transformers (ViT-B/16, ViT-L/16). For each metric, we highlight the best method in bold: higher values are better for ADCC, Coh, Inc, and ADD; lower values are better for AD, Com, and Time.

Group	**Approach**	**ADCC↑**	**AD↓**	**Coh↑**	**Com↓**	**Inc↑**	**ADD↑**	**Time↓**
CNN Linear Heads	DD-CAM	**0.8087**	**0.1393**	**0.9877**	**0.2137**	**0.4673**	**0.3873**	1.5443
	Grad-CAM	0.8010	0.1530	0.9790	0.2517	0.4200	0.3823	0.0240
	Grad-CAM++	0.7727	0.1973	0.9857	0.2517	0.3590	0.3747	0.0290
	XGrad-CAM	0.8010	0.1537	0.9790	0.2517	0.3740	0.3823	**0.0140**
	Layer-CAM	0.7653	0.1980	0.9807	0.2450	0.3463	0.3500	0.0540
	Score-CAM	0.7937	0.1653	0.9450	0.2300	0.4370	0.3560	1.6460
	Ablation-CAM	0.7803	0.1920	0.9580	0.2197	0.4023	0.3627	1.5463
	Recipro-CAM	0.7649	0.1510	0.9680	0.2246	0.4254	0.3715	0.0145
CNN Non-Linear Heads	DD-CAM	**0.7873**	**0.1610**	**0.9827**	0.2170	**0.3120**	**0.5423**	1.8267
	Grad-CAM	0.7437	0.1913	0.9437	0.2567	0.2790	0.5120	0.0110
	Grad-CAM++	0.7303	0.2077	0.9737	0.2373	0.2727	0.5217	0.0210
	XGrad-CAM	0.7500	0.1927	0.9500	0.2547	0.3113	0.5110	**0.0107**
	Layer-CAM	0.7033	0.2993	0.9643	**0.1880**	0.2843	0.5070	0.1347
	Score-CAM	0.7740	0.1677	0.9450	0.2383	0.2560	0.5223	1.2117
	Ablation-CAM	0.7507	0.2413	0.9707	0.2253	0.2820	0.5397	1.1983
	Recipro-CAM	0.7754	0.1735	0.9755	0.2815	0.2929	0.4813	0.0184
Vision Transformers	DD-CAM	0.7985	**0.0832**	0.**9950**	0.2010	**0.6000**	**0.3975**	1.9301
	Grad-CAM	0.1721	0.7851	0.7244	0.2303	0.0929	0.1147	0.0415
	Grad-CAM++	0.1345	0.8563	0.7479	0.1498	0.1000	0.0793	0.0260
	XGrad-CAM	0.2190	0.8611	0.7331	**0.1182**	0.0200	0.0549	**0.0238**
	Layer-CAM	0.1831	0.9468	0.7704	0.1930	0.0150	0.1491	0.0273
	ScoreCAM	**0.8103**	0.1529	0.8641	0.2883	0.2250	0.2028	4.0864
	Ablation-CAM	0.4389	0.5615	0.8280	0.1618	0.3000	0.1062	6.9294
	Recipro-CAM	0.6889	0.1381	0.9850	0.5351	0.4250	0.3954	0.0704

4.6 Experimental Design

We evaluate our approach on vision models to answer two key research questions:

Table 4.2: RQ2 Results: Average localization performance on 1,000 chest X-rays (DenseNet-121). Best values are in bold (higher is better for IoU, Precision, Recall; lower for Regions).

Approach	IoU ↑	Precision ↑	Recall ↑	Regions ↓
DD-CAM	**0.263**	**0.307**	**0.692**	**1.00**
GradCAM	0.060	0.088	0.204	1.41
GradCAM++	0.135	0.171	0.519	1.11
XGrad-CAM	0.049	0.081	0.161	1.39
LayerCAM	0.137	0.183	0.567	1.10
Score-CAM	0.055	0.093	0.149	1.02
Ablation-CAM	0.059	0.086	0.191	1.32
Recipro-CAM	0.181	0.251	0.568	1.37

RQ1 (Faithfulness): How do the generated saliency maps reflect the model's decision-making process, and how does our approach compare with state-of-the-art CAM methods across architectures?

RQ2 (Localization): How do saliency maps align with human-annotated regions of interest, and how does our approach compare with state-of-the-art methods in localization accuracy?

4.6.1 Datasets

For RQ1, we use 2,000 randomly sampled images from the ILSVRC2012 validation set [100], resized to 224×224 and normalized following standard ImageNet preprocessing. For RQ2, we use 1,000 chest radiographs from NIH ChestX-ray14 [127] together with the radiologist-annotated bounding boxes provided by the dataset. This allows the assessment of how well the saliency maps align with clinically relevant regions identified by human experts.

4.6.2 Models

We evaluate eight architectures spanning convolutional and transformer paradigms.

CNNs with linear classifier heads. ResNet-50 [50], EfficientNet-B0 [119], and Inception-v3 [117] use global average pooling followed by a single fully connected layer, where each feature map contributes additively to the final prediction. For such models, the representational units are linearly independent, allowing an optimized version of delta debugging. When contributions are additive, each unit can be tested once to determine whether it is necessary for preserving the prediction, reducing the worst-case complexity from $O(n^2)$ to $O(n)$. This optimization is implemented by (a) fixing the granularity to $n_{\text{init}} = K$ (each unit tested individually) and (b) proceeding sequentially without restarting after each removal.

CNNs with non-linear classifier heads. VGG-11, VGG-16, and VGG-19 [110] contain multiple fully connected layers with ReLU activations, introducing nonlinear dependencies among the final-layer feature maps. For these interacting units, we employ the standard delta debugging procedure, starting with coarse subdivisions ($n_{\text{init}} = 2$) and recursively refining the granularity as needed.

Vision Transformers. ViT-B/16 and ViT-L/16 [37] divide images into 16×16 patches, producing 196 patch tokens plus one CLS token. Although the classifier head is linear, the CLS token is computed after the final transformer block, where all patches interact through multi-head self-attention. Consequently, ViTs exhibit interacting units for sufficiency testing, and we configure delta debugging with coarse initial granularity ($n_{\text{init}} = 2$), refining it recursively as needed.

All models use TorchVision pretrained weights [122].

Medical Imaging Model. For the ChestX-ray14 localization experiments (RQ2), we use a DenseNet-121 model pretrained on large-scale chest X-ray datasets [32].

4.6.3 Baselines

We compare our approach to six commonly used CAM-based attribution approaches: Grad-CAM [104], Grad-CAM++ [25], XGrad-CAM [41], Layer-CAM [56], Ablation-CAM [35], and Score-CAM [126]. These approaches differ in how they estimate feature importance—via gradients, multi-layer aggregation, ablations, or activation perturbations—but all operate on the final-layer representation and therefore provide a consistent comparison framework. For ViTs, all baselines operate on patch tokens, following standard practice. Implementations are based on the PyTorch-CAM library [45]. We restrict comparisons to CAM-style approaches because perturbation-based metrics are not directly compatible with input-level approaches such as LIME [95] or SHAP [76].

4.6.4 Evaluation Metrics

For RQ1, we evaluate faithfulness on ImageNet using six standard perturbation-based metrics [25, 57, 90]: *Average Drop (AD, ↓)* quantifies the confidence decrease when replacing the input with the saliency map; *Coherency (Coh, ↑)* measures consistency between full-image and saliency map confidences; *Complexity (Com, ↓)* captures explanation sparsity via the L1 norm of the normalized heatmap; *Average DCC (ADCC, ↑)* summarizes AD, Coh, and Com as a harmonic mean; *Increase in Confidence (IC, ↑)* counts cases where the saliency map increases confidence; and *Average Drop in Deletion (ADD, ↑)* measures confidence decreases when masking highlighted regions, reflecting feature criticality.

For RQ2, we evaluate localization quality against radiologist-annotated bounding boxes using *IoU (↑)*, *Precision (↑)*, *Recall (↑)*, *Percentage Highlighted (↓)*, and *Number of Regions (↓)* [38, 35, 126]. IoU, precision, and recall measure region overlap, correctness of highlighted pixels, and coverage of the pathology, respectively, while the latter two quantify explanation focus and spatial coherence. All experiments use PyTorch on a 12 GB GPU system with 32 GB RAM, and results are averaged over five runs.

4.7 Results and Discussion

We evaluate DD-CAM against state-of-the-art CAM-based approaches to assess faithfulness and localization quality.

4.7.1 Faithfulness Evaluation (RQ1)

Across all model families, DD-CAM produces the most faithful and compact saliency maps. Averaged over three CNNs with linear heads, DD-CAM achieves the highest ADCC (0.8087), lowest Average Drop (0.1393), highest Coherency (0.9877), and lowest Complexity (0.2137), indicating that the highlighted regions both preserve model confidence and remain highly focused. Similar trends appear for nonlinear CNNs, where DD-CAM again achieves the highest ADCC (0.7873), lowest AD (0.1610), and strongest causal relevance (ADD = 0.5423), despite the feature interactions introduced by nonlinear classifier heads. Layer-CAM achieves a slightly lower Complexity (0.1880) in this group, though at the cost of reduced causal faithfulness (lower ADCC and higher Average Drop).

For Vision Transformers, DD-CAM achieves the lowest AD (0.0832), highest Increase in Confidence (0.600), and highest ADD (0.3975), showing that the regions corresponding to the minimal token set remain essential despite self-attention mixing. Score-CAM yields a slightly higher ADCC (0.8103) but produces substantially larger regions and requires 2.4× more computation, resulting in worse sparsity and efficiency. XGrad-CAM attains the lowest Complexity (0.1182) but exhibits weak causal alignment, as shown by its lower ADCC and higher Average Drop values.

In terms of runtime, gradient-based (e.g., Grad-CAM, XGrad-CAM) approaches are faster. DD-CAM remains efficient among gradient-free approaches like Score-CAM and Ablation-CAM by operating only on the final representational layer instead of requiring full forward pass.

Figure 4.1 shows saliency map comparisons for representative samples, illustrating how DD-CAM's minimal decision-preserving unit selection reduces clutter and yields clearer, more focused visualizations.

4.7.2 Localization (RQ2)

On ChestX-ray14, DD-CAM produces regions that closely align with radiologist-annotated regions. As shown in Table 4.2, it achieves the highest IoU (0.263), precision (0.307), and recall (0.692), outperforming the strongest baseline by approximately +45%, +22%, and +22%, respectively. Relative to the widely used Grad-CAM, these gains are even more pronounced (4.4× in IoU and 3.5× in precision), indicating substantially improved localization fidelity.

DD-CAM also yields the most compact explanations, averaging only 1.00 region per image, whereas baselines highlight 1.02–1.41 regions. This reduced fragmentation is evident in Figure 4.2, where DD-CAM isolates a single coherent pathological focus, while other methods yield diffuse or fragmented activations. Overall, DD-CAM provides the most human-aligned and semantically meaningful localizations among CAM-based methods.

4.8 Limitations

Although DD-CAM mitigates key limitations of CAM-based explanations—most notably by selecting minimal, decision-preserving feature map subsets—it still shares some constraints of the CAM

paradigm. Like other CAM-based approaches, DD-CAM saliency maps by upsampling the selected feature maps to match the input resolution. This post-processing step can introduce spatial imprecision, potentially affecting the granularity of highlighted regions. We mitigate this by focusing only on the most critical feature maps. Furthermore, our approach is inherently white-box, requiring access to internal activations and weights.

4.9 Conclusion

We introduce the first approach that applies the principle of minimal sufficiency to internal representations to produce minimal sufficient explanations for vision models in a representation-agnostic manner. Specifically, we ask which subset of representational units (feature maps for CNNs, patch tokens for ViTs) is jointly sufficient to preserve the model's predictions. By adapting software debugging strategies, the framework provides formal 1-minimality guarantees for the resulting explanations, ensuring that no redundant representational units remain. Empirically, DD-CAM improves faithfulness across the majority of model–metric combinations and enhances localization. **Future directions:** (1) Using minimal unit sets for model debugging and bias analysis; (2) Extending the framework to other domains by redefining units.

Acknowledgment

This work is supported by a research grant (70NANB21H092) from the Information Technology Laboratory of the National Institute of Standards and Technology (NIST).

Disclaimer: Certain equipment, instruments, software, or materials are identified in this paper in order to specify the experimental procedure adequately. Such identification is not intended to imply recommendation or endorsement of any product or service by NIST, nor is it intended to imply that the materials or equipment identified are necessarily the best available for the purpose.

4.10 Appendix: Proof of Proposition 1

Proposition 1 (Exclusion of non-positive-margin units)**.** *Let* $\hat{c} = \arg\max_c y_c(U)$*, and assume* $y_{\hat{c}}(U) > y_c(U)$ *for all* $c \neq \hat{c}$ *(i.e., the prediction is unambiguous). If unit* k *satisfies* $(w_{\hat{c},k} - w_{c,k})\, a_k \leq 0$ *for every* $c \neq \hat{c}$*, then for every sufficient set* $S \ni k$*, the set* $S \setminus \{k\}$ *is also sufficient.*

Proof. In a GAP+FC model, the margin between $\hat{c}$ and any alternative class c under subset S is

$$y_{\hat{c}}(S) - y_c(S) = \sum_{j \in S} (w_{\hat{c},j} - w_{c,j})\, a_j + (b_{\hat{c}} - b_c).$$

Let $S' = S \setminus \{k\}$. Removing unit k gives

$$y_{\hat{c}}(S') - y_c(S') = \big[y_{\hat{c}}(S) - y_c(S)\big] - (w_{\hat{c},k} - w_{c,k})\, a_k.$$

Since $(w_{\hat{c},k} - w_{c,k})\, a_k \leq 0$ by assumption, subtracting it can only increase or preserve the margin. Therefore $y_{\hat{c}}(S') - y_c(S') \geq y_{\hat{c}}(S) - y_c(S) > 0$ for all $c \neq \hat{c}$, and S' is sufficient. $\square$

Remark. When the final convolutional layer is followed by ReLU, all pooled activations satisfy $a_k \geq 0$. The condition then reduces to checking $w_{\hat{c},k} \leq w_{c,k}$ for all $c \neq \hat{c}$, a property of the weight matrix alone, independent of the input image.

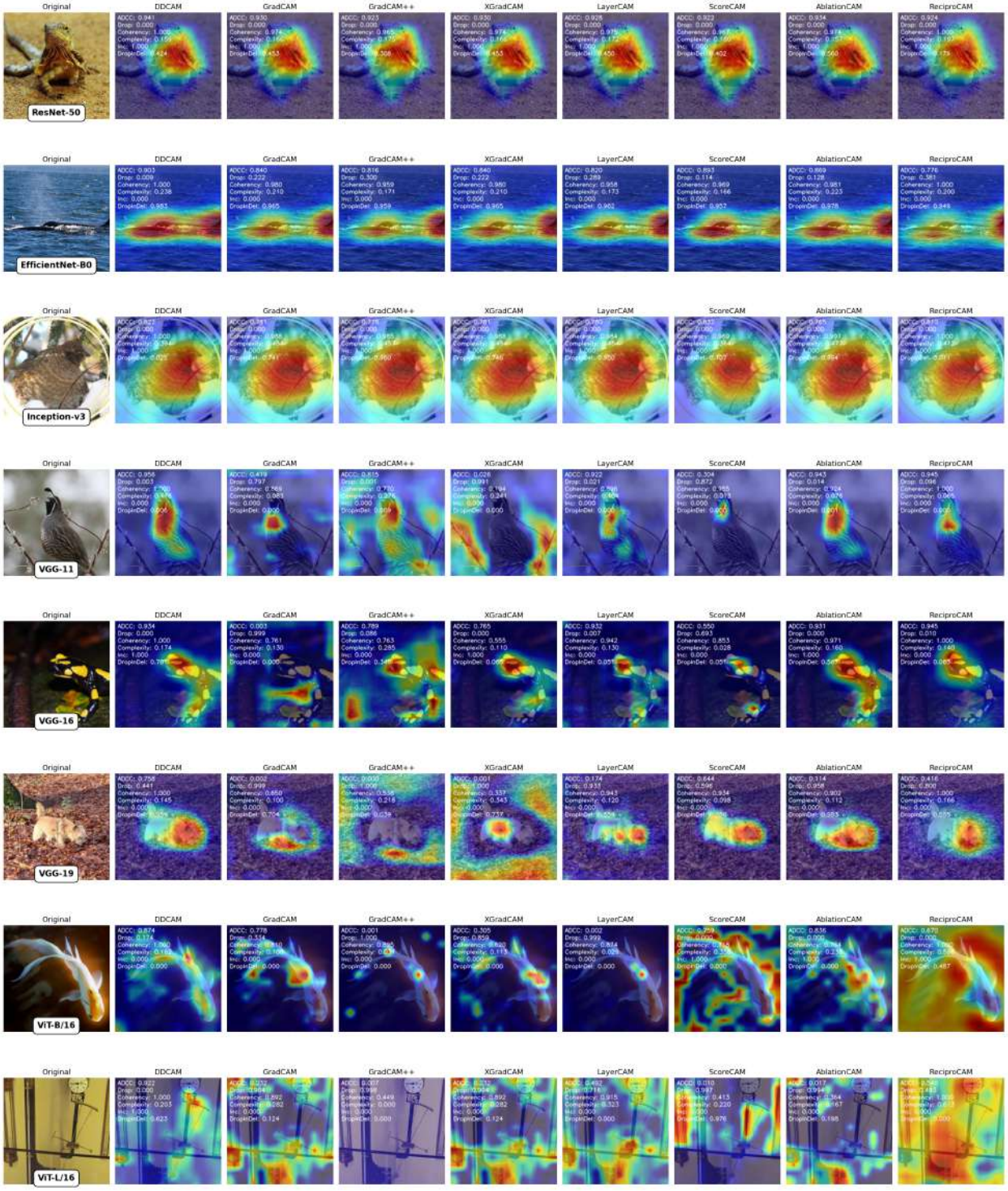

Figure 4.1: Qualitative comparison of saliency maps generated by our approach (DD-CAM) and baseline approaches (Grad-CAM, Grad-CAM++, XGrad-CAM, Layer-CAM, Score-CAM, Ablation-CAM, and Recipro-CAM).

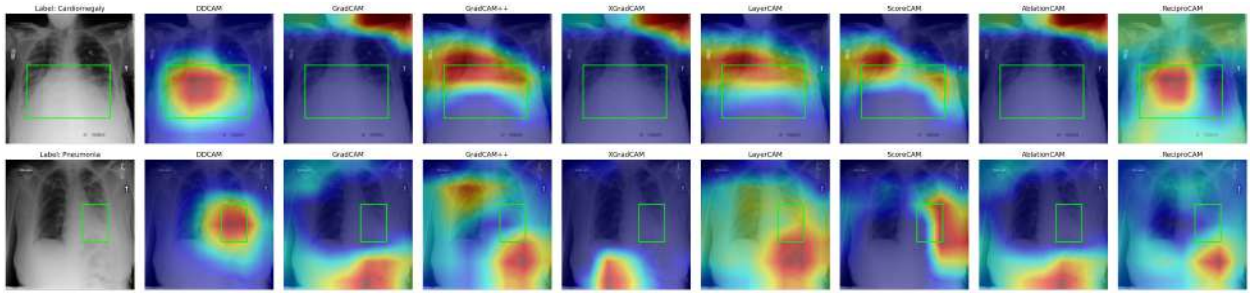

Figure 4.2: Qualitative localization examples on NIH ChestX-ray14. DD-CAM consistently isolates a single, compact pathological region while baselines vary between diffuse and fragmented responses.

Chapter 5

Explanation-Guided Model Repair

This chapter is based on: "Explanation-Guided Model Repair," targeting AAAI 2027.

5.1 Introduction

Chapters 2–4 address understanding and explaining model predictions. This chapter addresses the next logical question: once we can explain what drives failures, how do we fix them? Spurious correlations, where models exploit contextual features that correlate with class labels but do not generalize, are a major source of systematic failures [42]. Many strong repair methods either require group labels or rely on auxiliary supervision/proxy signals for identifying minority groups [71, 102]. EGMR operates without group labels, attribute annotations, or environment metadata, using concept profiling [15] and difference saliency to automatically surface and repair failure patterns. By generating additional concept-class interactions in the under-represented cells of the joint distribution, EGMR diffuses the shortcuts the model was relying on and pushes it toward relying on essential concepts instead.

EGMR connects to the few-feature insight: just as predictions depend on few features (Chapter 2) and explanations depend on few representational units (Chapter 4), systematic failures often manifest through a small number of recurring failure-associated concepts. Targeting these recurring concepts enables efficient repair.

5.2 Problem Setup

We consider an image classification setting with a trained classifier $f : \mathcal{X} \to \mathcal{Y}$, a training dataset $D_{\text{train}} = \{(x_i, y_i)\}$, and no access to group labels or annotations of spurious attributes. The classifier achieves high average accuracy but suffers systematic errors on minority subpopulations, that is, instances where the true class co-occurs with atypical contextual features (e.g., waterbirds on land backgrounds [103]). These errors arise because the training distribution contains statistical shortcuts: contextual features that correlate with class labels in the majority of training examples but do not generalize.

Scope of applicability. We assume the classifier has learned useful object-level features but also exploits distribution-linked contextual shortcuts for a subset of predictions. This is the common setting studied in the spurious correlation literature: the model is not completely broken, but it over-relies on background or context when those cues conflict with the true class. In the extreme case where the model has never learned essential features (predicting entirely from shortcuts), our repair strategy is insufficient, as the model has no useful representation to recover after the shortcut is removed.

Goal. Given only the trained classifier, its training data, and a validation set, identify recurring failure modes attributable to distributional shortcuts and repair them through targeted data augmentation, improving worst-group robustness while preserving performance on correctly classified instances. Mispredictions are collected from a held-out validation split (not the training set), as training-set errors from an ERM model [124] may not reflect test-time failure patterns due to overfitting.

Notation. Let $Y = \{c_1, c_2\}$ denote the class set. Let X_c^+ denote images correctly predicted as class c, and X_c^- denote images mispredicted as class c (i.e., images whose true class is not c but which the model assigns to c). We write $S_c(x)$ for the saliency map [110] of image x with respect to class c, and $w(x)$ for the SpLiCE [15] concept weight vector of x over a vocabulary V.

5.2.1 Key Insight

Shortcuts come from the training data: certain background or contextual features tend to appear together with a particular class label in most examples, so the model picks up on them. The model learns to use these shortcuts because they work well on average.

When a shortcut conflicts with the true class, the model fails. Looking at recurring failures together reveals the combinations the training data rarely or never provided: the right object in an unusual context.

Because we cannot perfectly tell essential features apart from spurious ones without outside information, our goal is not exact identification but to combine several simple checks that focus repair on features that are likely contextual while avoiding damage to features that define the class. Concretely, our goal is targeted sampling around mispredictions: we inpaint the failure-inducing margin region to produce more samples in that part of the input space, so the model stops relying on shortcuts and instead looks to real class-defining concepts when making predictions. The pipeline below progressively narrows down to features that are safe to edit, and confirms this by checking that repair improves accuracy on failing groups without hurting accuracy on the rest.

A central challenge in shortcut identification is telling apart features that define a class from contextual features that just happen to correlate with class labels. We observe that this distinction can be approximated from the training data alone, without assumptions about cause and effect: features that appear in only one class are more likely to define that class, while features that appear in multiple classes at different rates are more likely to be contextual (they can show up with any class, but happen to be unevenly distributed in the training data). We call the former **class-**

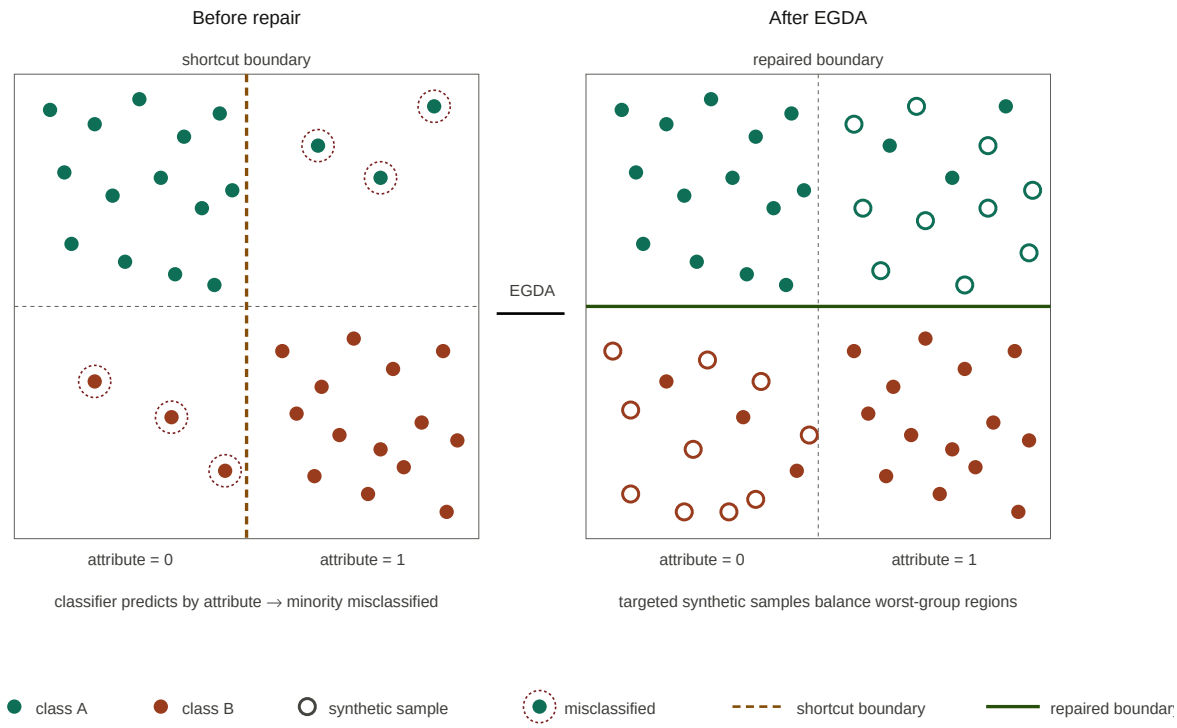

Figure 5.1: Targeted EGMR edits move the decision boundary, fixing minority-group errors. Before repair, the classifier separates points along the shortcut attribute (vertical dashed boundary), so minority-group instances (circled in red) fall on the wrong side. EGMR inpaints the failure-inducing margin region to add synthetic samples (open circles) in the under-represented worst-group cells, balancing the joint distribution. After fine-tuning, the decision boundary aligns with the true class-defining concept (horizontal solid line) rather than the shortcut attribute.

exclusive features and the latter **class-shared** features. Class-shared features are the natural candidates for shortcuts, because a feature concentrated in a single class is more likely to belong to that class than to be a side effect of how the dataset was collected.

Our method splits the set of concepts into these two groups, then uses average concept patterns from correctly classified examples to find which class-shared concepts are most unevenly distributed, and difference saliency maps to identify which of those concepts are actually driving individual failures. We then create the missing combinations by editing only the class-shared regions of failing images, regions that push the model toward the wrong prediction but not the correct one. This split also helps keep the true label intact: because class-shared content is not tied to a single class, editing it is unlikely to change the label. After fine-tuning on these augmented examples, we check whether the model's reliance on shortcuts goes down and accuracy on the worst-performing group goes up.

5.3 Method

EGMR operates in four stages: concept bank construction, failure pattern discovery, two-gate failure localization, and targeted synthetic augmentation. The only inputs are the trained classifier, its training data, a held-out validation set for collecting mispredictions, and access to a vision-language model and an image editor; no group labels are required at any stage. A concept-region pair (v, r) is a *recurrent repair target*, and forwarded to augmentation, only if it satisfies four criteria established progressively across the stages: (i) v is class-shared (Stage 1); (ii) v is asymmetric across class profiles (Stage 2); (iii) on each misprediction, r drives the wrong-class margin and decomposes into v (Stage 3); and (iv) the (v, r) pattern recurs across mispredictions (Stage 3).

5.3.1 Stage 1: Concept Bank Construction

The first stage splits the concept set into two groups (class-exclusive and class-shared) based on each concept's distribution across classes. Only class-shared concepts can be targeted for repair.

Class concept profiles. For each class c, we compute the average concept breakdown over correctly predicted images. We use SpLiCE [15] to break each image's CLIP [92] embedding into a sparse weight vector $w(x)$ over a concept set V, giving a class concept profile:

$$P_c = \frac{1}{|X_c^+|} \sum_{x \in X_c^+} w(x).$$

Because the model is accurate on average, X_c^+ is dominated by majority-group images (e.g., waterbirds on water), so P_c captures the concept distribution where the model gets things right, including its biases.

Exclusivity scoring. For each concept $v \in V$, we compute an exclusivity ratio:

$$E(v) = \frac{\min(P_{c_1}(v), P_{c_2}(v))}{\max(P_{c_1}(v), P_{c_2}(v)) + \epsilon}.$$

$E(v)$ near 0 means the concept appears almost only in one class (class-exclusive); $E(v)$ noticeably above zero means it appears in both (class-shared). We apply a threshold τ_E to split the vocabulary:

$$V_{\text{exclusive}} = \{v : E(v) < \tau_E\}, \quad V_{\text{shared}} = \{v : E(v) \geq \tau_E\}.$$

Concepts below a minimum frequency threshold ($\max(P_{c_1}(v), P_{c_2}(v)) < \tau_p$) are dropped before splitting to avoid noise from concepts that barely appear in either profile.

Rationale. A concept that appears in only one class may be class-defining ("swan" for waterbirds) or just a dataset artifact (a watermark, sensor bias), and editing it carries the highest risk of changing the true label, so we exclude all such concepts from repair as a precaution. Concepts that appear in both classes are less likely to define either: "forest" shows up with both landbirds and waterbirds, and the fact that it is skewed toward landbirds is a property of the dataset rather

than of what makes a bird a landbird, so editing it is unlikely to flip the label. The split also avoids a failure mode of pure skew-based detection, where perfectly skewed class-exclusive concepts ("cardinal," "pelican") would dominate the ranking ahead of the actual shortcut concepts ("forest," "water"). Joint patterns where two individually class-shared concepts together produce a class-exclusive signal are not captured by this per-concept split and remain a current limitation.

5.3.2 Stage 2: Failure Pattern Discovery

Within V_{shared}, we rank concepts by the normalized disparity:

$$A(v) = \frac{|P_{c_1}(v) - P_{c_2}(v)|}{\max(P_{c_1}(v), P_{c_2}(v)) + \epsilon}.$$

High $A(v)$ marks the strongest shortcut candidates: concepts that appear in both classes (so they are unlikely to define either) but are heavily concentrated in one (so the model can lean on them as a cue). Low-asymmetry shared concepts like "bird" are not targeted. The result is a candidate list of population-level shortcut hypotheses; Stage 3 validates them against actual errors. Restricting asymmetry scoring to V_{shared} keeps perfectly skewed class-exclusive concepts ("cardinal," "pelican") out of the ranking.

5.3.3 Stage 3: Failure Localization and Two-Gate Validation

The third stage validates Stage 2 candidates with two complementary gates. Only candidates passing both are promoted to recurrent repair targets and forwarded to Stage 4.

Difference saliency. For each mispredicted image, we compute saliency maps with respect to the predicted (wrong) and true classes, then take the rectified difference:

$$D = \text{ReLU}(S_{\text{wrong}} - S_{\text{true}}).$$

D highlights regions whose local effect pushes the prediction toward the wrong class. We do not treat D as causal proof, since saliency maps are imperfect [6]; it supplies a candidate region whose causal role is checked downstream by Stage 4 (does removing the region change the prediction?) and by cross-image recurrence.

Instance-alignment gate. We crop the diff-map region, embed it with CLIP [92], and decompose it with SpLiCE [15]. The misprediction passes if the cropped region's dominant concepts match the high-asymmetry shared concepts from Stage 2 (compared by ranking, not exact weights, with crops padded to a minimum size) *and* those dominant concepts lie in V_{shared} rather than $V_{\text{exclusive}}$.

Cross-instance recurrence gate. The instance-alignment gate must be passed by at least a fraction α of mispredictions for the affected class ($\alpha = 0.5$ in our experiments); an isolated alignment is not promoted. The two gates are complementary: per-image evidence prevents accepting population-level coincidences, and recurrence prevents accepting instance-level flukes.

5.3.4 Stage 4: Targeted Synthetic Augmentation

Stage 4 rebalances the joint distribution $P(v, c)$ by generating synthetic samples that fill the under-represented cell flagged by high $A(v)$. In Waterbirds [103], $P(\text{water} \mid \text{landbird}) \ll P(\text{water} \mid \text{waterbird})$ and symmetrically, so the model exploits the near-perfect background-to-label link as a shortcut. Filling both unusual cells, (landbird, water) and (waterbird, land), with valid examples reduces the imbalance and removes the cue.

Synthesis procedure. For each seed image (mispredictions that passed both gates of Stage 3, not arbitrary training images, so that synthesis stays coupled to the diagnosis):

1. Use the diff-map mask from Stage 3 as the inpainting region.
2. Inpaint variations using a diffusion model [98] with prompts targeting the under-populated cell (e.g., a landbird in water-leaning backgrounds). The bird and label are kept by design; only the contextual region is edited.

Because Stage 1 restricts repair targets to V_{shared} and Stage 3 confirms the masked region's dominant concepts are class-shared per image, edits operate on content that is, by distribution, not class-defining. This is a safeguard, not a guarantee.

Fine-tuning. Synthetic samples are mixed with the original training data; the mixing ratio trades off repair aggressiveness against stability. We track three quantities: (a) clean in-distribution accuracy, (b) accuracy on originally correct images, and (c) accuracy on the worst-performing group. The first two test for collateral damage; the third tests whether augmentation actually rebalances the joint distribution.

Figure 5.2 illustrates the end-to-end pipeline on a single mispredicted image from the validation set, walking through the diagnose, localize, and repair stages.

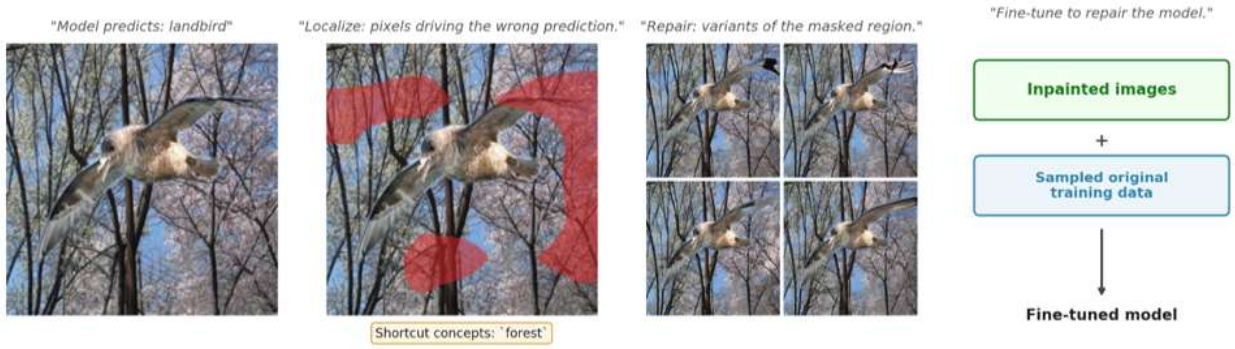

Figure 5.2: End-to-end EGMR pipeline on one mispredicted validation-set image. Original image: model predicts landbird. **Diagnose** flags '*forest*' as the suspect concept. **Localize** masks the forest region. **Repair** generates diffusion variants and fine-tunes.

5.3.5 Scope

EGMR targets the partial-learning setting; if the model predicts entirely from shortcuts, the detection stages remain valid but repair requires explicit supervision rather than augmentation alone. The framework is instantiated with specific tools (SpLiCE [15] for concepts, Grad-CAM [104] for saliency, Stable Diffusion [98] for inpainting), but the core logic (class-shared/class-exclusive partition, asymmetry within the shared bank, two-gate validation, and targeted augmentation) does not depend on these choices.

5.4 Preliminary Results

5.4.1 Dataset

We evaluate EGMR on Waterbirds [102], a standard benchmark for spurious correlation studies in image classification. Each image is labeled either landbird or waterbird, and photographed against either a land or water background. The dataset is constructed so that the bird's class is strongly correlated with its background context in the training distribution, which gives the classifier a statistical shortcut to exploit.

The training set contains 4,795 images split across four groups defined by the (class, background) pair. The distribution is severely imbalanced (Table 5.1).

Table 5.1: Waterbirds training distribution by (class, background) group.

Group	Class	Background	Train images	Fraction
0	landbird	land	3,498	73.0%
1	landbird	water	184	3.8%
2	waterbird	land	56	1.2%
3	waterbird	water	1,057	22.0%

The two minority groups (landbird on water, waterbird on land) together account for only 5.0% of the training data. A model that learns to associate background with class achieves high average training accuracy but fails systematically on these two minority groups. The waterbird-on-land group, with 56 training images, is the most extreme case.

The test set contains 5,794 images, with group sizes 2,255/2,255/642/642, decoupling test-set group balance from training-set imbalance. Worst-group test accuracy therefore reflects the model's ability to generalize beyond the training shortcut.

5.4.2 Baselines

We compare EGMR against two baselines, all using the same ResNet-50 [50] backbone initialized from ImageNet-pretrained weights:

- **ERM** [124]. Standard empirical risk minimization. ERM is the baseline most exposed to the spurious correlation, since no mechanism in its loss function encourages the model to disentangle

bird identity from background context.

- **JTT (Just Train Twice)** [71]. A representative no-group-labels baseline. JTT trains an ERM model in a first pass, collects the training examples it misclassifies, and retrains in a second pass with those misclassified examples upweighted. JTT itself does not claim to identify minority groups; it simply upweights mispredictions. Like EGMR, it does not require group annotations.

We report two metrics: overall test accuracy (standard measure, weighted by group size) and worst-group test accuracy (the lowest per-group accuracy, which captures generalization to under-represented subpopulations).

5.4.3 Results

Table 5.2: Worst-group and overall test accuracy on Waterbirds.

Method	Worst-group acc.	Overall acc.
ERM [124]	72.59%	87.97%
JTT [71]	80.37%	92.41%
EGMR (ours)	**87.54%**	**95.62%**

EGMR substantially outperforms both baselines on both metrics. Compared to ERM, EGMR improves worst-group accuracy by 14.95 percentage points (from 72.59% to 87.54%) and overall accuracy by 7.65 percentage points (from 87.97% to 95.62%). Compared to JTT, EGMR improves worst-group accuracy by 7.17 percentage points and overall accuracy by 3.21 percentage points. These gains are obtained without group labels, attribute annotations, or any auxiliary supervision beyond the trained ERM model and the training images.

5.5 Remaining Tasks

Remaining work:

1. Evaluation on additional datasets: CelebA [73], ImageNet-9 [133], and ColorMNIST [10].
2. Ablation studies on each module (concept bank partition, asymmetry scoring, difference saliency, cross-instance recurrence, targeted synthetic augmentation).
3. Comparison with additional baselines DFR [59], DISC [132], and Correct-N-Contrast [138].
4. Performance retention on originally correct instances.

Target submission: AAAI 2027, September 2026.

5.6 Conclusion

EGMR closes the loop from explanation to action. By identifying recurring explanation-derived failure patterns and repairing them through targeted augmentation, it demonstrates how explanations can serve as practical tools for model improvement.

Chapter 6

Conclusion and Timeline

6.1 Summary of Contributions

This dissertation makes the following contributions to machine learning interpretability:

C1. Empirical validation that most model predictions depend on interactions among fewer than five features, providing the foundation for minimal explanations and targeted repair (Chapter 2, IWCT 2024).

C2. A novel approach to local explanations using adversarial pairs that bracket the decision boundary, achieving state-of-the-art fidelity (16/18 configurations) and stability (Chapter 3, KDD 2026).

C3. Adaptation of delta debugging to identify 1-minimal sufficient subsets of representational units for vision model explanation, outperforming baselines in 15/18 faithfulness evaluations and improving medical imaging localization by 45% (Chapter 4, targeting NeurIPS 2026).

C4. An explanation-guided framework for repairing recurring failure-driving patterns without group labels, achieving 87.54% worst-group accuracy on Waterbirds (Chapter 5, targeting AAAI 2027).

Together, these contributions advance ML reliability by moving from understanding to explanation to repair, demonstrating that targeting the critical structures underlying predictions—whether features, decision boundaries, representational units, or spurious concepts—yields both better explanations and more robust models.

6.2 Timeline

Table 6.1: Proposed Timeline toward Dissertation Defense.

Period	**Milestone**
Apr – May 2026	DD-CAM: Submit to NeurIPS 2026
Apr – May 2026	Dissertation: Chapter 6 (Conclusion), revision, committee review
May – Aug 2026	Complete EGMR
Jun – Jul 2026	Final revisions based on committee feedback
Nov 2026	Dissertation defense
Sep 2026	EGMR: Submit to AAAI 2027

Bibliography

[1] Dd-cam: Anonymous code release. Anonymous release.

[2] Mushroom. UCI Machine Learning Repository, 1981.

[3] Mushroom. UCI Machine Learning Repository, 1987. DOI: 10.24432/C5959T.

[4] Able: Adversarially bracketed local explanation. Anonymous software release, 2025.

[5] Samira Abnar and Willem Zuidema. Quantifying attention flow in transformers. *arXiv preprint arXiv:2005.00928*, 2020.

[6] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. *Advances in neural information processing systems*, 31, 2018.

[7] Muhammad Aurangzeb Ahmad, Carly Eckert, and Ankur Teredesai. Interpretable machine learning in healthcare. In *Proc. 2018 ACM Int. Conf. Bioinformatics, Computational Biology, and Health Informatics*, pages 559–560, 2018.

[8] David Alvarez-Melis and Tommi S. Jaakkola. On the robustness of interpretability methods. *arXiv preprint arXiv:1806.08049*, 2018.

[9] Sercan Ö. Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. In *AAAI*, volume 35, pages 6679–6687, 2021.

[10] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.

[11] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, et al. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115, 2020.

[12] David Baehrens, Timon Schroeter, Stefan Harmeling, Motoaki Kawanabe, Katja Hansen, and Klaus-Robert Müller. How to explain individual classification decisions. *Journal of Machine Learning Research*, 11:1803–1831, 2010.

[13] Barry Becker and Ronny Kohavi. Adult. UCI Machine Learning Repository, 1996. DOI: 10.24432/C5XW20.

[14] Barry Becker and Ronny Kohavi. Adult. UCI Machine Learning Repository, 1996.

[15] Usha Bhalla, Alex Oesterling, Suraj Srinivas, Flavio P Calmon, and Himabindu Lakkaraju. Interpreting clip with sparse linear concept embeddings (splice). *Advances in Neural Information Processing Systems*, 37:84298–84328, 2024.

[16] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[17] Jock Blackard. Covertype. UCI Machine Learning Repository, 1998.

[18] Marko Bohanec. Car evaluation. UCI Machine Learning Repository, 1988.

[19] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[20] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[21] Renee C. Bryce, Yu Lei, D. Richard Kuhn, and Raghu Kacker. Combinatorial testing. In *Handbook of Research on Software Engineering and Productivity Technologies: Implications of Globalization*, pages 196–208. IGI Global, 2010.

[22] Seok-Yong Byun and Wonju Lee. ViT-ReciproCAM: gradient and attention-free visual explanations for vision transformer. *arXiv preprint arXiv:2310.02588*, 2023.

[23] Brandon Carter, Jonas Mueller, Siddhartha Jain, and David Gifford. What made you do this? understanding black-box decisions with sufficient input subsets. In *AISTATS*, pages 567–576, 2019.

[24] Jaganmohan Chandrasekaran, Yu Lei, Raghu Kacker, and D. Richard Kuhn. A combinatorial approach to testing deep neural network-based autonomous driving systems. In *2021 IEEE Int. Conf. Software Testing, Verification and Validation Workshops (ICSTW)*, pages 57–66, 2021.

[25] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N. Balasubramanian. Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks. In *2018 IEEE WACV*, pages 839–847, 2018.

[26] Hila Chefer, Shir Gur, and Lior Wolf. Transformer interpretability beyond attention visualization. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, pages 782–791, 2021.

[27] Jianbo Chen, Michael I. Jordan, and Martin J. Wainwright. HopSkipJumpAttack: A query-efficient decision-based attack. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1277–1294, 2020.

[28] Jianbo Chen, Le Song, Martin Wainwright, and Michael Jordan. Learning to explain: An information-theoretic perspective on model interpretation. In *International Conference on Machine Learning*, 2018.

[29] Jianyu Chen, Shengbo Eben Li, and Masayoshi Tomizuka. Interpretable end-to-end urban autonomous driving with latent deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, 23(6):5068–5078, 2021.

[30] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[31] Martina Cinquini and Riccardo Guidotti. Causality-aware local interpretable model-agnostic explanations. In *World Conference on Explainable Artificial Intelligence*, pages 108–124, 2024.

[32] Joseph Paul Cohen, Joseph D. Viviano, Paul Bertin, Paul Morrison, Parsa Torabian, Matteo Guarrera, Matthew P. Lungren, Akshay Chaudhari, Rupert Brooks, Mohammad Hashir, et al. TorchXRayVision: A library of chest x-ray datasets and models. In *International Conference on Medical Imaging with Deep Learning*, pages 231–249, 2022.

[33] Nelson Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

[34] David R. Cox. The regression analysis of binary sequences (with discussion). *Journal of the Royal Statistical Society, Series B*, 20(2):215–242, 1958.

[35] Saurabh Desai and Harish Guruprasad Ramaswamy. Ablation-CAM: Visual explanations for deep convolutional network via gradient-free localization. In *Proc. IEEE/CVF WACV*, pages 983–991, 2020.

[36] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[37] Alexey Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[38] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[39] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177):1–81, 2019.

[40] Ruth C. Fong and Andrea Vedaldi. Interpretable explanations of black boxes by meaningful perturbation. In *Proc. IEEE Int. Conf. Computer Vision*, pages 3429–3437, 2017.

[41] Ruigang Fu, Qingyong Hu, Xiaohu Dong, Yulan Guo, Yinghui Gao, and Biao Li. Axiom-based Grad-CAM: Towards accurate visualization and explanation of CNNs. *arXiv preprint arXiv:2008.02312*, 2020.

[42] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[43] Amirata Ghorbani, Abubakar Abid, and James Zou. Interpretation of neural networks is fragile. In *International Conference on Machine Learning*, 2019.

[44] Sushmito Ghosh and Douglas L. Reilly. Credit card fraud detection with a neural-network. In *Proc. 27th Hawaii Int. Conf. System Sciences*, volume 3, pages 621–630, 1994.

[45] Jacob Gildenblat and contributors. Pytorch library for cam methods, 2021.

[46] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[47] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2015.

[48] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):1–42, 2018.

[49] T. Hastie and R. Tibshirani. *Generalized Additive Models*. Chapman and Hall/CRC, 1990.

[50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[51] Marti A. Hearst, Susan T. Dumais, Edgar Osuna, John Platt, and Bernhard Scholkopf. Support vector machines. *IEEE Intelligent Systems and their Applications*, 13(4):18–28, 1998.

[52] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning Workshop*, 2015.

[53] Mark Holmstrom, Dylan Liu, and Christopher Vo. Machine learning applied to weather forecasting. *Meteorol. Appl.*, 10:1–5, 2016.

[54] Andreas Holzinger, Chris Biemann, Constantinos S. Pattichis, and Douglas B. Kell. What do we need to build explainable AI systems for the medical domain? *arXiv preprint arXiv:1712.09923*, 2017.

[55] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. TabTransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678*, 2020.

[56] Peng-Tao Jiang, Chang-Bin Zhang, Qibin Hou, Ming-Ming Cheng, and Yunchao Wei. LayerCAM: Exploring hierarchical class activation maps for localization. *IEEE Transactions on Image Processing*, 30:5875–5888, 2021.

[57] Hyungsik Jung and Youngrock Oh. Towards better explanations of class activation mapping. In *Proc. IEEE/CVF Int. Conf. Computer Vision*, pages 1336–1344, 2021.

[58] Krishna Khadka, Jaganmohan Chandrasekaran, Yu Lei, Raghu N. Kacker, and D. Richard Kuhn. Synthetic data generation using combinatorial testing and variational autoencoder. In *2023 IEEE Int. Conf. Software Testing, Verification and Validation Workshops (ICSTW)*, pages 228–236, 2023.

[59] Polina Kirichenko, Pavel Izmailov, and Andrew Gordon Wilson. Last layer re-training is sufficient for robustness to spurious correlations. *arXiv preprint arXiv:2204.02937*, 2022.

[60] Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*, pages 1885–1894, 2017.

[61] Boris Kovalerchuk, Evgenii Vityaev, Alexandr Demin, and Adam Wilinski. Interpretable machine learning for financial applications. In *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*, pages 721–749. Springer, 2023.

[62] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 2012.

[63] D. Richard Kuhn, Dolores R. Wallace, and Albert M. Gallo. Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6):418–421, 2004.

[64] Max Kuhn and Kjell Johnson. *Applied Predictive Modeling.* Springer, 2013.

[65] S. M. Fazle Rabby Labib, Joyanta Jyoti Mondal, Meem Arafat Manab, Xi Xiao, and Sarfaraz Newaz. Tailoring adversarial attacks on deep neural networks for targeted class manipulation using DeepFool algorithm. *Scientific Reports*, 15(1):10790, 2025.

[66] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[67] Y. G. Lee, J. Y. Oh, D. Kim, and G. Kim. Shap value-based feature importance analysis for short-term load forecasting. *Journal of Electrical Engineering & Technology*, 18(1):579–588, 2023.

[68] Pantelis Linardatos, Vasilis Papastefanopoulos, and Sotiris Kotsiantis. Explainable AI: A review of machine learning interpretability methods. *Entropy*, 23(1):18, 2020.

[69] Zachary C. Lipton. The mythos of model interpretability. In *ICML Workshop on Human Interpretability in Machine Learning*, 2016.

[70] Zachary C. Lipton. The mythos of model interpretability. *Queue*, 16(3):31–57, 2018.

[71] Evan Z Liu, Behzad Haghgoo, Annie S Chen, Aditi Raghunathan, Pang Wei Koh, Shiori Sagawa, Percy Liang, and Chelsea Finn. Just train twice: Improving group robustness without training group information. In *International Conference on Machine Learning*, pages 6781–6792. PMLR, 2021.

[72] Yuchuang Liu, Zhishuai Liu, Xiaolei Luo, and Haifang Zhao. Diagnosis of parkinson's disease based on shap value feature selection. *Biocybernetics and Biomedical Engineering*, 42(3):856–869, 2022.

[73] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of the IEEE international conference on computer vision*, pages 3730–3738, 2015.

[74] Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. Consistent individualized feature attribution for tree ensembles. *arXiv preprint arXiv:1802.03888*, 2018.

[75] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

[76] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

[77] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.

[78] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018.

[79] R. Manjula, S. Jain, S. Srivastava, and P. R. Kher. Real estate value prediction using multivariate regression models. In *IOP Conf. Series: Materials Science and Engineering*, volume 263, page 042098, 2017.

[80] Xinlei Mi, Bo Zou, Fengzhu Zou, and Jun Hu. Permutation-based identification of important biomarkers for complex diseases via machine learning models. *Nature Communications*, 12(1):3008, 2021.

[81] George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81, 1956.

[82] Christoph Molnar. *Interpretable machine learning: A guide for making black box models explainable*. Leanpub, 2020.

[83] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. DeepFool: a simple and accurate method to fool deep neural networks. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 2574–2582, 2016.

[84] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.

[85] Stefano Nembrini, Inke R. König, and Marvin N. Wright. The revival of the gini importance? *Bioinformatics*, 34(21):3711–3718, 2018.

[86] Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, Ian Molloy, and Ben Edwards. Adversarial robustness toolbox v1.2.0. *CoRR*, 1807.01069, 2018.

[87] Ankita Ramjibhai Patel, Jaganmohan Chandrasekaran, Yu Lei, Raghu N. Kacker, and D. Richard Kuhn. A combinatorial approach to fairness testing of machine learning models. In *2022 IEEE Int. Conf. Software Testing, Verification and Validation Workshops (ICSTW)*, pages 94–101, 2022.

[88] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *IEEE Int. Conf. Data Science and Advanced Analytics (DSAA)*, pages 399–410, 2016.

[89] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[90] Samuele Poppi, Marcella Cornia, Lorenzo Baraldi, and Rita Cucchiara. Revisiting the evaluation of class activation mapping for explainability: A novel metric and experimental analysis. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, pages 2299–2304, 2021.

[91] J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.

[92] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.

[93] Gopi Krishnan Rajbahadur, Shaowei Wang, Gustavo Ansaldi Oliva, Yasutaka Kamei, and Ahmed E. Hassan. The impact of feature importance methods on the interpretation of defect classifiers. *IEEE Transactions on Software Engineering*, 48(7):2245–2261, 2021.

[94] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Model-agnostic interpretability of machine learning. *arXiv preprint arXiv:1606.05386*, 2016.

[95] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?" explaining the predictions of any classifier. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[96] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?" explaining the predictions of any classifier. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[97] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Anchors: High-precision model-agnostic explanations. In *AAAI*, volume 32, 2018.

[98] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[99] Cynthia Rudin. Stop explaining black box machine learning models for high-stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

[100] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[101] Hamid Saadatfar, Zeinab Kiani-Zadegan, and Benyamin Ghahremani-Nezhad. US-LIME: Increasing fidelity in LIME using uncertainty sampling on tabular data. *Neurocomputing*, 597:127969, 2024.

[102] Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731*, 2019.

[103] Shiori Sagawa, Aditi Raghunathan, Pang Wei Koh, and Percy Liang. An investigation of why overparameterization exacerbates spurious correlations. In *International Conference on Machine Learning*, pages 8346–8356. PMLR, 2020.

[104] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proc. IEEE Int. Conf. Computer Vision*, pages 618–626, 2017.

[105] Ali Shafahi, Mahyar Najibi, Mohammad Amin Ghiasi, Zheng Xu, John Dickerson, Christoph Studer, Larry S. Davis, Gavin Taylor, and Tom Goldstein. Adversarial training for free! *Advances in Neural Information Processing Systems*, 32, 2019.

[106] Rushdi Shams and Robert E. Mercer. Classifying spam emails using text and readability features. In *2013 IEEE 13th Int. Conf. Data Mining*, pages 657–666, 2013.

[107] Sharath M. Shankaranarayana and Davor Runje. ALIME: Autoencoder based approach for local interpretability. In *IDEAL*, pages 454–463, 2019.

[108] Lloyd S. Shapley. A value for n-person games. *Contributions to the Theory of Games*, 2(28):307–317, 1953.

[109] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.

[110] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[111] Pushpa Singh, Narendra Singh, Krishna Kant Singh, and Akansha Singh. Diagnosing of disease using machine learning. In *Machine Learning and the Internet of Medical Things in Healthcare*, pages 89–111. Academic Press, 2021.

[112] Dylan Slack, Sophie Hilgard, Emily Jia, Sameer Singh, and Himabindu Lakkaraju. Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods. In *Advances in Neural Information Processing Systems*, 2020.

[113] Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. SmoothGrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*, 2017.

[114] Fengxi Song, Zhongwei Guo, and Dayong Mei. Feature selection using principal component analysis. In *2010 Int. Conf. System Science, Engineering Design and Manufacturing Informatization*, volume 1, pages 27–30, 2010.

[115] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, and Martin Riedmiller. Striving for simplicity: The all convolutional net. In *ICLR Workshop Track*, 2015.

[116] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning*, pages 3319–3328, 2017.

[117] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[118] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

[119] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pages 6105–6114, 2019.

[120] Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. *Introduction to Data Mining*. Addison-Wesley, 2005.

[121] Zeren Tan, Yang Tian, and Jian Li. Glime: general, stable and local lime explanation. *Advances in Neural Information Processing Systems*, 36, 2024.

[122] TorchVision maintainers and contributors. Torchvision: Pytorch's computer vision library, 2016.

[123] Berk Ustun and Cynthia Rudin. Supersparse linear integer models for optimized medical scoring systems. *Machine Learning*, 102(3):349–391, 2016.

[124] Vladimir Vapnik. Principles of risk minimization for learning theory. *Advances in neural information processing systems*, 4, 1991.

[125] Fulton Wang and Cynthia Rudin. Falling rule lists. In *Advances in Neural Information Processing Systems*, 2015.

[126] Haofan Wang, Zifan Wang, Mengnan Du, Fan Yang, Zijian Zhang, Sirui Ding, Piotr Mardziel, and Xia Hu. Score-CAM: Score-weighted visual explanations for convolutional neural networks. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops*, pages 24–25, 2020.

[127] Xiaosong Wang, Yifan Peng, Le Lu, Zhiyong Lu, Mohammadhadi Bagheri, and Ronald M. Summers. ChestX-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 2097–2106, 2017.

[128] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[129] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer wisconsin (diagnostic). UCI Machine Learning Repository, 1993.

[130] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer wisconsin (diagnostic). UCI Machine Learning Repository, 1995. DOI: 10.24432/C5DW2B.

[131] Svante Wold, Kim Esbensen, and Paul Geladi. Principal component analysis. *Chemometrics and Intelligent Laboratory Systems*, 2(1-3):37–52, 1987.

[132] Shirley Wu, Mert Yuksekgonul, Linjun Zhang, and James Zou. Discover and cure: Concept-aware mitigation of spurious correlation. In *International Conference on Machine Learning*, pages 37765–37786. PMLR, 2023.

[133] Kai Xiao, Logan Engstrom, Andrew Ilyas, and Aleksander Madry. Noise or signal: The role of image backgrounds in object recognition. In *International Conference on Learning Representations (ICLR)*, 2021.

[134] I-Cheng Yeh. Default of credit card clients. UCI Machine Learning Repository, 2009.

[135] Muhammad Rehman Zafar and Naimul Mefraz Khan. DLIME: A deterministic local interpretable model-agnostic explanations approach for computer-aided diagnosis systems. *arXiv preprint arXiv:1906.10263*, 2019.

[136] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.

[137] Brian Hu Zhang, Blake Lemoine, and Margaret Mitchell. Mitigating unwanted biases with adversarial learning. In *Proc. 2018 AAAI/ACM Conf. AI, Ethics, and Society*, pages 335–340, 2018.

[138] Michael Zhang, Nimit S Sohoni, Hongyang R Zhang, Chelsea Finn, and Christopher Ré. Correct-n-contrast: A contrastive approach for improving robustness to spurious correlations. *arXiv preprint arXiv:2203.01517*, 2022.

[139] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 586–595, 2018.

[140] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pages 2921–2929, 2016.